



SECURITY REVIEW · PREPARED FOR

Riverboat

Mainnet Alpha v1.0

DATE	CONDUCTED BY	COMMIT	REMEDIATION
19 June 2026	Kyan Sharma, Security Researcher Zuhaib Mohammed, Security Researcher	a4ccdc6	4897aaf

Contents

1	About Slot Zero Security	3
2	Disclaimer	3
3	System Overview	3
3.1	Technical Overview	4
3.2	Overall Security Posture	4
3.3	Post-Remediation Notes	5
4	Executive Summary	5
4.1	Overview	5
4.2	Timeline	5
4.3	Scope	5
4.4	Issues Found	6
4.5	Findings Summary	6
5	Findings	7
	Medium Severity	7
	[M-01] Existing Wagers Accept Participants After Contract Result Is Known . . .	7
	[M-02] payout_data lacks provenance between provisional risk and finalized payouts	16
	[M-03] Child wagers depend on closable and reinitializable parent contracts . . .	22
	[M-04] Child Manual Push ignores the parent contract's late-result challenge window	27
	Low Severity	30
	[L-01] set_contract_result Allows Resolving Unfinalized Contracts	30
	[L-02] close_contract Can Leave an Existing Terms Account Stranded	35
	[L-03] Incomplete Native Account Close Sequence Leaves Closed PDAs Program- Owned and Allocated	39
	[L-04] locked_seats bitmap is not cleared when locked seats become paid or closed	41
	[L-05] Non-Canonical seat_count Accounting Can Desynchronize Wager State From Actual Seat PDA Indices	52
	[L-06] Incomplete seat state reset on open_seat and take_seat leaves stale resolution_data	55
	[L-07] close_seat sends another participant's seat rent to the wager creator during delayed cleanup	57

Informational Severity	61
[I-01] write_contract Allows Blank Contract Headlines	61
[I-02] open_seat relies on unchecked lamport arithmetic for stake refunds	62
[I-03] Account discriminator is checked after full Borsh deserialization	63
[I-04] Arbitrary seat authority can leave seat PDAs uncleared during cleanup . .	64
[I-05] Creator-only append_terms overwrite does not clear stale raw terms bytes	66
[I-06] set_contract_result can mark a contract resolved with a Pending result . .	70

1 About Slot Zero Security

Slot Zero Security is a smart contract security team focused on practical, adversarial review of crypto systems. Our researchers combine manual code analysis, state-machine reasoning, targeted proof-of-concept testing, and remediation support to identify vulnerabilities that matter in production. We work closely with engineering teams to validate fixes, communicate risk clearly, and help protocols ship with stronger security assumptions.

2 Disclaimer

This report was prepared for the Riverboat Mainnet Alpha v1.0 snapshot. Security reviews are time-boxed and scope-bound. They reduce risk by identifying issues visible within the reviewed snapshot and validating remediations reviewed during the engagement, but they cannot prove the absence of all vulnerabilities. This report should be read as a point-in-time assessment of the scoped code and reviewed remediation snapshot.

AI-assisted tooling was used to support report preparation, formatting, code navigation, and review workflow organization. All security findings, severity assessments, remediation validation statements, and final report conclusions were reviewed and approved by the named human security researchers.

3 System Overview

The reviewed Riverboat Mainnet Alpha v1.0 snapshot is a native Solana Rust protocol for binary outcome wagers. This report is not a full restatement of the May 2026 baseline review; it focuses on the pre-remediation Phase 2 diff and the validated remediation that followed. The original Phase 2 diff added or modified result-setting, challenge-window, parent-child settlement, seat replacement, payout, and close-path behavior. During remediation, Riverboat removed contract-level result state and the contract result-setting path from the latest MVP design.

The baseline protocol shape remains the same, but Phase 2 meaningfully expanded the lifecycle surface around already-created contracts, wagers, and seats. Where this section refers to contract-result behavior, it describes the original reviewed diff; the findings below document whether that behavior was fixed, removed, or carried forward in the final remediated snapshot. The relevant v1.0 security model is therefore centered on how these account types move from creation to result, payout, reopening, and cleanup:

- i) **Contracts and terms** were reviewed for finalization, contract-result timestamping, close, and terms-overwrite behavior that downstream wagers could depend on. The final remediated MVP removes contract-level result state and `set_contract_result`.
- ii) **Wagers** are the active settlement surface in the latest design, including wager-level result-setting, manual push timing, payout, and close behavior. The review also evaluated the original parent-result timing assumptions where they affected downstream wagers.
- iii) **Seats** now have a wider lifecycle: open, taken, deposited, predicted, locked, resolved, paid, reopened, or closed. The review focused on whether stale seat fields can affect later settlement or cleanup.
- iv) **Vault and native accounts** custody lamports across deposit, payout, close, refund-like

cleanup, and rent-drain paths.

Because the codebase is native Solana rather than Anchor, these properties depend directly on explicit account ownership checks, PDA derivation, signer validation, discriminator checks, and state-transition rules in the processors. The Phase 2 changes were reviewed primarily for places where a newly added lifecycle path could bypass an invariant that was previously only exercised by wager creation, seat locking, or simple payout flows.

3.1 Technical Overview

The Mainnet Alpha v1.0 implementation remains split across two native Solana programs with shared state and helper logic in `protocol`. For this Phase 2 review, the most relevant implementation changes and remediation-sensitive surfaces were:

- i) **Contract program:** pre-remediation contract result behavior and close behavior around `close_contract`, including how finalization, contract-level `result`, contract-level `result_set_at`, terms accounts, and closed/reinitialized parent accounts affected downstream settlement assumptions. The latest MVP removes `set_contract_result` and the contract-level result fields.
- ii) **Wager program:** new or materially changed flows around `set_wager_result`, `open_seat`, `close_seat`, `close_wager`, `set_seat_resolution`, and `claim_payout`, plus existing participation handlers that now need to respect settled wager state.
- iii) **Protocol state and engine logic:** Phase 2 fields and calculations around wager-level `result_set_at`, `payout_data`, `locked_seats`, seat indexes, payout/risk domain separation, and resolution-window arithmetic. Removed contract-level result fields are discussed in the affected findings.

The primary assumed attacker is an adversarial wager creator or participant interacting through normal Solana transactions. Validator-level ordering, frontend compromise, off-chain services, infrastructure, production key custody, and deployment administration were not treated as primary attack surfaces for this scoped Phase 2 diff.

3.2 Overall Security Posture

Before Remediation

In the pre-remediation snapshot, the main risk concentration was in new Phase 2 state transitions rather than the baseline wager creation flow. The highest-value issues came from lifecycle joins: existing wagers accepting participants after a parent result, child wagers depending on mutable or closable parent contracts, Manual Push using the wrong challenge-window start, provisional payout data being reused as finalized payout data, and seat or account cleanup paths leaving stale state behind.

After Remediation

Latest-code remediation review was performed against `4897aaf`. The submitted report findings have no remaining open items: sixteen findings were fixed and validated, and one residual risk

around `locked_seats` cleanup was acknowledged by the Riverboat team.

3.3 Post-Remediation Notes

The remediation review validated the Phase 2 fixes against `4897aaf`. Detailed recommendations are tracked in the individual findings and corresponding team responses; this section does not introduce additional open remediation items.

At the close of this review, the only residual behavior carried forward is the acknowledged `locked_seats` cleanup behavior in L-04.

4 Executive Summary

4.1 Overview

The review covered the Riverboat Mainnet Alpha v1.0 Phase 2 diff at commit `a4ccdc6`. The engagement is scoped to newly pushed changes in `contract_program`, `wager_program`, and security-relevant shared logic in `protocol`. The reviewed value-moving flow remains binary wagers funded in lamports, with Phase 2 adding result-setting, challenge-window, seat replacement, close, and custody-drain behavior.

4.2 Timeline

Initial review window	4 June to 17 June 2026
Report date	19 June 2026
Review type	Manual Phase 2 diff review
Release name	Mainnet Alpha v1.0
Remediation Review	Performed against latest reviewed code <code>4897aaf</code> ; 16 findings fixed, 1 acknowledged, 0 open
Snapshot	<code>a4ccdc66b0300a3908f9ef94e89b47a88b194caf</code>
Remediation	<code>4897aaf13f222d9d9a0aafcb59dfa31729b144d1</code>

4.3 Scope

In scope	The Phase 2 diff in <code>contract_program</code> , <code>wager_program</code> , and security-relevant shared state and engine logic in <code>protocol</code> .
Primary focus	Result setting, challenge windows, parent-child wager settlement, seat replacement and cleanup, payout claiming, close flows, and native lamport custody paths.
Handler coverage	All net-new or materially changed Phase 2 processors were reviewed, including result setters, close processors, seat open/close/reuse flows, deposit and prediction flows, seat resolution, locking, terms updates, and payout claiming.
Out of scope	Frontend code, off-chain services, infrastructure, deployment administration, unchanged baseline behavior except where needed for invariant tracing, and non-operational token path stubs.

4.4 Issues Found

Severity	Total	Fixed and Validated	Acknowledged	Open
Critical	0	0	0	0
High	0	0	0	0
Medium	4	4	0	0
Low	7	6	1	0
Informational	6	6	0	0

Status definitions:

Fixed and Validated: Remediation was reviewed against the latest reviewed code and relevant fix behavior was validated.

Acknowledged: the Riverboat team accepted the issue or residual risk, but it was not fully remediated in this snapshot.

Open: the issue remained unresolved at the time of this report.

4.5 Findings Summary

ID	Severity	Title	Status
M-01	Medium	Existing Wagers Accept Participants After Contract Result Is Known	Fixed and Validated
M-02	Medium	<code>payout_data</code> lacks provenance between provisional risk and finalized payouts	Fixed and Validated
M-03	Medium	Child wagers depend on closable and reinitializable parent contracts	Fixed and Validated
M-04	Medium	Child Manual Push ignores the parent contract's late-result challenge window	Fixed and Validated
L-01	Low	<code>set_contract_result</code> Allows Resolving Unfinalized Contracts	Fixed and Validated
L-02	Low	<code>close_contract</code> Can Leave an Existing Terms Account Stranded	Fixed and Validated
L-03	Low	Incomplete Native Account Close Sequence Leaves Closed PDAs Program-Owned and Allocated	Fixed and Validated
L-04	Low	<code>locked_seats</code> bitmap is not cleared when locked seats become paid or closed	Acknowledged
L-05	Low	Non-Canonical <code>seat_count</code> Accounting Can Desynchronize Wager State From Actual Seat PDA Indices	Fixed and Validated
L-06	Low	Incomplete seat state reset on <code>open_seat</code> and <code>take_seat</code> leaves stale <code>resolution_data</code>	Fixed and Validated
L-07	Low	<code>close_seat</code> sends another participant's seat rent to the wager creator during delayed cleanup	Fixed and Validated
I-01	Informational	<code>write_contract</code> Allows Blank Contract Headlines	Fixed and Validated
I-02	Informational	<code>open_seat</code> relies on unchecked lamport arithmetic for stake refunds	Fixed and Validated
I-03	Informational	Account discriminator is checked after full Borsh deserialization	Fixed and Validated

I-04	Informational	Arbitrary seat authority can leave seat PDAs uncleared during cleanup	Fixed and Validated
I-05	Informational	Creator-only <code>append_terms</code> overwrite does not clear stale raw terms bytes	Fixed and Validated
I-06	Informational	<code>set_contract_result</code> can mark a contract resolved with a Pending result	Fixed and Validated

5 Findings

Medium Severity

[M-01] Existing Wagers Accept Participants After Contract Result Is Known

The wager program correctly blocks new wager creation once the parent contract has been resolved, but it does not consistently block participation in wagers that already exist.

The root cause is that the parent contract's resolved state is only checked during `make_wager`. After a wager has been created, the main participation instructions rely on the wager's local `lock_by` deadline and either do not check `contract.resolved` or do not load the contract account at all.

This means that if the parent contract result becomes known before an existing wager's `lock_by`, users may still be able to join or finalize their position in that wager with knowledge of the outcome.

Affected participation paths include:

- `take_seat`: loads the contract, but does not reject `contract.resolved`
- `process_deposit`: does not load the contract account
- `set_prediction_data`: does not load the contract account
- `lock_seat`: does not load the contract account

As a result, the protocol prevents new wagers from being opened after contract resolution, but existing wagers can still accept economically meaningful participation until their own `lock_by` deadline.

Impact

A participant can wait until the contract result is already public, then join or finalize an existing wager before `wager.lock_by`.

This creates unfair post-result participation and can cause bounded loss to earlier participants who committed stake or predictions before the result was known. The issue is most impactful when an existing wager still has an available seat, reserved seat, or staked-but-unlocked seat after the parent contract result has been set.

The impact depends on the wager state:

- If a seat is still available, an attacker can take the seat after the result is known.
- If the attacker already has a reserved seat, they can wait to deposit until after the result is

known.

- If the attacker already deposited but has not set a prediction, they can choose the winning prediction after the result is known.
- If the attacker has a staked seat with a prediction but has not locked, they can update the prediction before `lock_by` and then lock.
- If all seats were already locked before the result became known, this specific attack path is not available because normal prediction updates are no longer allowed.

Proof of Concept

Example: Assume today is June 11, 2026.

Alice creates a binary contract:

"Will Elon Musk tweet about Dogecoin by December 31, 2026?"

The contract has:

- `resolves_by` = December 31, 2026
- `outcome 0` = No
- `outcome 1` = Yes

Bob creates a two-seat wager on this contract:

- `stake` = 10 SOL
- `lock_by` = December 20, 2026

Charlie joins early. He takes seat 0, deposits 10 SOL, and predicts No.

On August 1, 2026, Elon Musk tweets about Dogecoin. The contract resolution authority sets the parent contract result to Yes.

The outcome is now known on-chain, but Bob's wager is still before its `lock_by` date. Eve observes the resolved contract result and then joins the existing wager. She takes the remaining seat, deposits stake, sets her prediction to Yes, and locks her seat.

These actions succeed because the wager participation flow checks only `wager.lock_by`, not whether the parent contract has already been resolved.

Run: `cargo test -p wager_program --test test_post_resolution_participation -- --nocapture`

```

use borsh::{BorshDeserialize, to_vec};

use protocol::discriminator::CONTRACT_DISCRIMINATOR;
use protocol::ids::{CONTRACT_PROGRAM_ID, SEAT_WAGER_PROGRAM_ID};
use protocol::state::{
    Contract,
    Content,
    Currency,
    EventResult,
    Judgment,
    OutcomeType,
    Prediction,
    ResolutionMechanism,
    Wager,
};

use wager_program::dto::{
    MakeWagerData,
    SeatDepositData,
    SeatPredictionData,
    TakeSeatData,
};
use wager_program::instruction::WagerInstruction;

use litesvm::LiteSVM;
use solana_address::Address;
use solana_sdk::{
    account::Account,
    instruction::{AccountMeta, Instruction},
    message::Message,
    signature::{Keypair, Signer},
    transaction::Transaction,
};
use solana_sdk_ids::system_program;
use solana_sysvar::clock::Clock;

fn send_tx(
    svm: &mut LiteSVM,
    payer: &Keypair,
    signers: &[&Keypair],
    ix: Instruction,
) -> bool {
    let message = Message::new(&[ix], Some(&payer.pubkey()));
    let tx = Transaction::new(signers, message, svm.latest_blockhash());
    svm.send_transaction(tx).is_ok()
}

#[test]
fn existing_wager_accepts_participation_after_contract_result_is_known() {
    let mut svm = LiteSVM::new();
    let program_id = SEAT_WAGER_PROGRAM_ID;

```

```

svm.add_program_from_file(program_id, "../target/deploy/wager_program.so")
    .unwrap();

let mut clock = Clock::default();

// June 11, 2026 00:00:00 UTC.
clock.unix_timestamp = 1_781_136_000;
svm.set_sysvar(&clock);

let contract_creator = Keypair::new();
let wager_creator = Keypair::new();
let charlie = Keypair::new();
let eve = Keypair::new();

svm.airdrop(&contract_creator.pubkey(), 50_000_000_000).unwrap();
svm.airdrop(&wager_creator.pubkey(), 50_000_000_000).unwrap();
svm.airdrop(&charlie.pubkey(), 50_000_000_000).unwrap();
svm.airdrop(&eve.pubkey(), 50_000_000_000).unwrap();

let contract_account = Keypair::new();
let wager_account = Keypair::new();

// December 31, 2026 00:00:00 UTC.
let contract_resolves_by = 1_798_675_200;

// December 20, 2026 00:00:00 UTC.
let wager_lock_by = 1_797_724_800;

let stake_amount = 10_000_000_000;

let contract = Contract {
    discriminator: CONTRACT_DISCRIMINATOR,
    creator: contract_creator.pubkey(),
    resolution_authority: contract_creator.pubkey(),
    created_at: clock.unix_timestamp,
    resolves_by: contract_resolves_by,
    resolved: false,
    private: false,
    finalized: true,
    has_terms: false,
    terms_bump: 255,
    category: [0u8; 8],
    hashtag: [0u8; 16],
    outcomes: OutcomeType::Binary,
    headline: Content::Plaintext(
        "Will Elon Musk tweet about Dogecoin by December 31, 2026?".to_string(),
    ),
    result: Some(Judgment::SingleOutcome(EventResult::Pending)),
    result_set_at: None,
};

let mock_contract_account = Account {
    lamports: 1_000_000_000,
    data: to_vec(&contract).unwrap(),
    owner: CONTRACT_PROGRAM_ID,
    executable: false,
    rent_epoch: 0,
};

```

```

svm.set_account(contract_account.pubkey(), mock_contract_account.into())
    .unwrap();

let (vault_pda, _) = Address::find_program_address(
    &[b"vault", &wager_account.pubkey().to_bytes()],
    &program_id,
);

let make_wager_ix = Instruction::new_with_bytes(
    program_id,
    &to_vec(&WagerInstruction::MakeWager {
        dto: MakeWagerData {
            seat_capacity: 2,
            stake: stake_amount,
            currency: Currency::Lamports,
            lock_by: wager_lock_by,
            private: false,
            resolution_mechanism: ResolutionMechanism::PeerConsensus,
            resolution_authority: None,
        },
    })
    .unwrap(),
    vec![
        AccountMeta::new(wager_creator.pubkey(), true),
        AccountMeta::new_readonly(system_program::id(), false),
        AccountMeta::new(contract_account.pubkey(), false),
        AccountMeta::new(wager_account.pubkey(), true),
        AccountMeta::new(vault_pda, false),
    ],
);

```

```

assert!(
    send_tx(
        &mut svm,
        &wager_creator,
        &[&wager_creator, &wager_account],
        make_wager_ix,
    ),
    "make_wager should succeed while the contract is unresolved"
);

let (charlie_seat, _) = Address::find_program_address(
    &[b"seat", &wager_account.pubkey().to_bytes(), &0u8.to_le_bytes()],
    &program_id,
);

let charlie_take_ix = Instruction::new_with_bytes(
    program_id,
    &to_vec(&WagerInstruction::TakeSeat {
        dto: TakeSeatData {
            authority: charlie.pubkey(),
        },
    })
    .unwrap(),
    vec![
        AccountMeta::new(charlie.pubkey(), true),
        AccountMeta::new_readonly(system_program::id(), false),
        AccountMeta::new(contract_account.pubkey(), false),
        AccountMeta::new(wager_account.pubkey(), false),
        AccountMeta::new(charlie_seat, false),
    ],
);

```

```

);

assert!(
    send_tx(&mut svm, &charlie, [&charlie], charlie_take_ix),
    "Charlie should be able to take the first seat before resolution"
);

let charlie_deposit_ix = Instruction::new_with_bytes(
    program_id,
    &to_vec(&WagerInstruction::ProcessDeposit {
        dto: SeatDepositData {
            amount: stake_amount,
        },
    })
    .unwrap(),
    vec![
        AccountMeta::new(charlie.pubkey(), true),
        AccountMeta::new_readonly(system_program::id(), false),
        AccountMeta::new(wager_account.pubkey(), false),
        AccountMeta::new(vault_pda, false),
        AccountMeta::new(charlie_seat, false),
    ],
);

assert!(
    send_tx(&mut svm, &charlie, [&charlie], charlie_deposit_ix),
    "Charlie should be able to deposit before resolution"
);

let charlie_prediction_ix = Instruction::new_with_bytes(
    program_id,
    &to_vec(&WagerInstruction::SetPredictionData {
        dto: SeatPredictionData {
            seat_index: 0,
            prediction: Prediction::Binary(0),
        },
    })
    .unwrap(),
    vec![
        AccountMeta::new(charlie.pubkey(), true),
        AccountMeta::new(wager_account.pubkey(), false),
        AccountMeta::new(charlie_seat, false),
    ],
);

assert!(
    send_tx(&mut svm, &charlie, [&charlie], charlie_prediction_ix),
    "Charlie predicts No before the result is known"
);

// Simulate the parent contract result becoming known on-chain:
// Elon Musk tweets about Dogecoin, so the result is Yes / outcome index 1.
let mut resolved_contract = contract;
resolved_contract.resolved = true;
resolved_contract.result = Some(Judgment::SingleOutcome(EventResult::Resolved(1)));
resolved_contract.result_set_at = Some(clock.unix_timestamp + 100);

```

```

let resolved_contract_account = Account {
  lamports: 1_000_000_000,
  data: to_vec(&resolved_contract).unwrap(),
  owner: CONTRACT_PROGRAM_ID,
  executable: false,
  rent_epoch: 0,
};

svm.set_account(contract_account.pubkey(), resolved_contract_account.into())
  .unwrap();

let second_wager_account = Keypair::new();
let (second_vault_pda, _) = Address::find_program_address(
  &[b"vault", &second_wager_account.pubkey().to_bytes()],
  &program_id,
);

let second_make_wager_ix = Instruction::new_with_bytes(
  program_id,
  &to_vec(&WagerInstruction::MakeWager {
    dto: MakeWagerData {
      seat_capacity: 2,
      stake: stake_amount,
      currency: Currency::Lamports,
      lock_by: wager_lock_by,
      private: false,
      resolution_mechanism: ResolutionMechanism::PeerConsensus,
      resolution_authority: None,
    },
  },
  })

  .unwrap(),
  vec![
    AccountMeta::new(wager_creator.pubkey(), true),
    AccountMeta::new_readonly(system_program::id(), false),
    AccountMeta::new(contract_account.pubkey(), false),
    AccountMeta::new(second_wager_account.pubkey(), true),
    AccountMeta::new(second_vault_pda, false),
  ],
);

assert!(
  !send_tx(
    &mut svm,
    &wager_creator,
    &[&wager_creator, &second_wager_account],
    second_make_wager_ix,
  ),
  "control check: new wager creation is correctly blocked after contract resolution"
);

let (eve_seat, _) = Address::find_program_address(
  &[b"seat", &wager_account.pubkey().to_bytes(), &1u8.to_le_bytes()],
  &program_id,
);

```

```

let eve_take_ix = Instruction::new_with_bytes(
    program_id,
    &to_vec(&WagerInstruction::TakeSeat {
        dto: TakeSeatData {
            authority: eve.pubkey(),
        },
    })
    .unwrap(),
    vec![
        AccountMeta::new(eve.pubkey(), true),
        AccountMeta::new_readonly(system_program::id(), false),
        AccountMeta::new(contract_account.pubkey(), false),
        AccountMeta::new(wager_account.pubkey(), false),
        AccountMeta::new(eve_seat, false),
    ],
);

assert!(
    send_tx(&mut svm, &eve, [&eve], eve_take_ix),
    "BUG: Eve can take a seat after the parent contract result is known"
);

```

```

let eve_deposit_ix = Instruction::new_with_bytes(
    program_id,
    &to_vec(&WagerInstruction::ProcessDeposit {
        dto: SeatDepositData {
            amount: stake_amount,
        },
    })
    .unwrap(),
    vec![
        AccountMeta::new(eve.pubkey(), true),
        AccountMeta::new_readonly(system_program::id(), false),
        AccountMeta::new(wager_account.pubkey(), false),
        AccountMeta::new(vault_pda, false),
        AccountMeta::new(eve_seat, false),
    ],
);

assert!(
    send_tx(&mut svm, &eve, [&eve], eve_deposit_ix),
    "BUG: Eve can deposit after the parent contract result is known"
);

```

```

let eve_prediction_ix = Instruction::new_with_bytes(
    program_id,
    &to_vec(&WagerInstruction::SetPredictionData {
        dto: SeatPredictionData {
            seat_index: 1,
            prediction: Prediction::Binary(100),
        },
    })
    .unwrap(),
    vec![
        AccountMeta::new(eve.pubkey(), true),
        AccountMeta::new(wager_account.pubkey(), false),
        AccountMeta::new(charlie_seat, false),
        AccountMeta::new(eve_seat, false),
    ],
);

assert!(
    send_tx(&mut svm, &eve, [&eve], eve_prediction_ix),
    "BUG: Eve can set the winning prediction after the result is known"
);

```

```

let eve_lock_ix = Instruction::new_with_bytes(
    program_id,
    &to_vec(&WagerInstruction::LockSeat).unwrap(),
    vec![
        AccountMeta::new(eve.pubkey(), true),
        AccountMeta::new(wager_account.pubkey(), false),
        AccountMeta::new(eve_seat, false),
    ],
);

assert!(
    send_tx(&mut svm, &eve, &[&eve], eve_lock_ix),
    "BUG: Eve can lock her seat after the result is known"
);

let wager_account_data = svm.get_account(&wager_account.pubkey()).unwrap();
let wager = Wager::deserialize(&mut &wager_account_data.data[..]).unwrap();

assert_eq!(wager.seat_count, 2);
assert_eq!(wager.locked_count(), 1);
}

```

Recommendation

Block post-result entry and commitment for existing wagers.

The fix should be applied to the instructions that let users create or change pre-result economic exposure:

- take_seat
- process_deposit
- set_prediction_data
- lock_seat

Each of these instructions should load the parent contract account, verify that the wager belongs to that contract, and reject the action if the parent contract result has already been set.

Example check:

```

if contract_account.owner != &CONTRACT_PROGRAM_ID {
    return Err(ProgramError::IncorrectProgramId);
}

let contract = Contract::from_account(contract_account)?;

if wager.contract != *contract_account.key {
    return Err(ProgramError::InvalidAccountData);
}

if contract.resolved || contract.result_set_at.is_some() {
    return Err(ProgramError::InvalidAccountData);
}

```

Team Response

Fixed and Validated. The Riverboat team removed contract-level result state from the latest design and now keeps settlement state on each `Wager`. Participation paths that can progress a wager now reject already-settled wagers through `wager.result.is_settled()`, so users can no

longer join or finalize a wager after that wager has been resolved.

[M-02] payout_data lacks provenance between provisional risk and finalized payouts

The program uses `seat.payout_data` for two different meanings:

- before settlement, it stores provisional risk computed from player predictions;
- after settlement, it stores the finalized amount that `claim_payout` pays.

The field carries no provenance or finalization marker, so instructions cannot distinguish "temporary risk math" from "value finalized by settlement." This creates two exploit classes from the same root cause: write-side mutation and read-side stale claiming.

First, `set_prediction_data` recomputes `payout_data` for every supplied seat once all current predictions are valid. It only checks that the caller's own target seat is still `Staked`, so a still-staked participant can cause another seat's `payout_data` to be recomputed after that other seat is already `Locked`.

Second, after Manual Push, the `claim_payout` status check accepts any seat that is not already `Paid` and pays the current `seat.payout_data`. The crux is that Manual Push finalizes payout data only for locked seats, but `claim_payout` later treats any non-`Paid` seat's current `payout_data` as payable. A funded `Staked` non-locked seat can therefore be marked `Paid` using stale provisional payout data. The normal refund path is then blocked, and leftover vault value can later be swept by the wager creator during `close_wager`.

Relevant code paths:

- `wager_program/src/processor.rs:420 set_prediction_data`
- `wager_program/src/processor.rs:503` only target seat status is checked
- `wager_program/src/processor.rs:520 / :530` recomputes payout data for seats
- `wager_program/src/processor.rs:741` settlement reads risk from `payout_data`
- `wager_program/src/processor.rs:765 / :766` peer settlement writes final payouts back into `payout_data`
- `wager_program/src/processor.rs:786 claim_payout`
- `wager_program/src/processor.rs:895` Manual Push via `set_wager_result`
- `wager_program/src/processor.rs:974` Manual Push writes locked-seat payouts
- `wager_program/src/processor.rs:1108 close_wager`
- `wager_program/src/processor.rs:1168 / :1184` vault close/sweep path
- `protocol/src/state.rs:122` default zero payout
- `protocol/src/state.rs:272` Payout is used for both risk and final payout

Impact

This can cause bounded wager-level accounting loss or payout mutation.

In the write-side variant, a participant can change another participant's economic state after that other participant believes their seat is locked. This locked-seat repricing path is the simpler and strongest variant. In the narrower post-settlement overwrite variant, a later claim drains

the overwritten payout amount rather than the original finalized payout.

The post-settlement overwrite path has a narrow setup: peer-consensus settlement happens before `lock_by`, the higher-index paid seat is closed, the zero-lamport account is purged or otherwise reinitialized, and that reused seat triggers prediction recomputation before the remaining locked seat claims. The simpler locked-seat repricing path does not need that close/reuse setup.

In the read-side variant, a participant can be marked as paid for a stale amount even though part of their stake remains in the vault. The included PoC has the stale seat authority call `claim_payout` directly. The same sink is reachable through the creator-rendered claim path in `claim_payout` after `result_set_at + RESOLUTION_WINDOW`: the creator can invoke the stale claim for the participant's seat, pay the stale amount to that participant, mark the seat `Paid`, and later close the wager to recover the stranded vault balance.

This is Medium severity because the impact changes claimable value inside affected wagers, but it remains bounded to the affected wager and depends on specific seat state or lifecycle conditions.

Proof of Concept

The following tests demonstrate the write-side and read-side variants. They use the same LiteSVM harness style as the existing Riverboat tests.

```
#[test]
fn rbp2_04_locked_seat_risk_can_be_repriced_by_other_staked_seat() {
    let mut h = setup_mock_wager(STAKE, NOW + 10_000, NOW + 1_000_000);
    prepare_two_staked_seats(&mut h);

    let creator = h.creator.insecure_clone();
    let taker = h.taker.insecure_clone();
    let seat0 = h.seat0;
    let seat1 = h.seat1;

    set_prediction(&mut h, &creator, 0, 60, &[seat0, seat1]);
    set_prediction(&mut h, &taker, 1, 40, &[seat0, seat1]);

    let locked_risk_before = payout(&h.svm, &seat0);
    lock_seat(&mut h, &creator, seat0);

    set_prediction(&mut h, &taker, 1, 99, &[seat0, seat1]);

    let seat = get_seat(&h.svm, &seat0);
    assert_eq!(seat.status, SeatStatus::Locked);
    assert_ne!(locked_risk_before, seat.payout_data.value().unwrap());
}
```

```

#[test]
fn rbp2_05_settled_high_index_seat_reuse_rewrites_existing_locked_payout() {
    let mut h = setup_mock_wager(STAKE, NOW + 10_000, NOW + 1_000_000);
    prepare_two_locked_seats(&mut h);

    let creator = h.creator.insecure_clone();
    let taker = h.taker.insecure_clone();
    let seat0 = h.seat0;
    let seat1 = h.seat1;

    set_resolution(&mut h, &creator, 0, resolved(1), &[seat0, seat1].unwrap());
    set_resolution(&mut h, &taker, 1, resolved(1), &[seat0, seat1].unwrap());

    let settled_payout_before = payout(&h.svm, &seat0);

    claim(&mut h, &taker, seat1, taker.pubkey()).unwrap();
    close_seat(&mut h, &taker, seat1, taker.pubkey()).unwrap();

    if let Some(closed_seat1) = h.svm.get_account(&seat1) {
        assert_eq!(closed_seat1.lamports, 0);
        assert!(closed_seat1.data.iter().all(|byte| *byte == 0));
    }

    purge_zero_lamport_account(&mut h.svm, seat1);
    retake_open_seat(&mut h, &taker, seat1, taker.pubkey(), taker.pubkey());
    deposit(&mut h, &taker, seat1, STAKE);
    set_prediction(&mut h, &taker, 1, 99, &[seat0, seat1]);

    let overwritten_payout = payout(&h.svm, &seat0);
    assert_ne!(settled_payout_before, overwritten_payout);

    let vault_before_claim = h.svm.get_account(&h.vault).unwrap().lamports;
    claim(&mut h, &creator, seat0, creator.pubkey()).unwrap();
    let vault_after_claim = h.svm.get_account(&h.vault).unwrap().lamports;

    assert_eq!(vault_before_claim - vault_after_claim, overwritten_payout);
    assert_ne!(vault_before_claim - vault_after_claim, settled_payout_before);
}

```

```

#[test]
fn rbp2_01_non_locked_stale_seat_can_claim_after_manual_push() {
    let contract_resolves_by = NOW + 20_000;
    let mut h = setup_mock_wager(STAKE, NOW + 10_000, contract_resolves_by);
    prepare_two_staked_seats(&mut h);

    let creator = h.creator.insecure_clone();
    let taker = h.taker.insecure_clone();
    let seat0 = h.seat0;
    let seat1 = h.seat1;

    set_prediction(&mut h, &creator, 0, 60, &[seat0, seat1]);
    set_prediction(&mut h, &taker, 1, 40, &[seat0, seat1]);

    let stale_payout = payout(&h.svm, &seat1);
    assert!(stale_payout > 0 && stale_payout < STAKE);

    lock_seat(&mut h, &creator, seat0);

    set_clock(&mut h.svm, contract_resolves_by + RESOLUTION_WINDOW + 1);
    set_wager_push(&mut h, &creator, &[seat0]).unwrap();

    claim(&mut h, &taker, seat1, taker.pubkey()).unwrap();

    let seat = get_seat(&h.svm, &seat1);
    assert_eq!(seat.status, SeatStatus::Paid);
    assert!(open_seat(&mut h, &taker, seat1, taker.pubkey()).is_err());
}

```

```

    claim(&mut h, &creator, seat0, creator.pubkey()).unwrap();
    close_seat(&mut h, &creator, seat0, creator.pubkey()).unwrap();
    close_seat(&mut h, &taker, seat1, taker.pubkey()).unwrap();

    let vault_before_close = h.svm.get_account(&h.vault).unwrap().lamports;
    assert!(vault_before_close >= STAKE - stale_payout);

    close_wager(&mut h).unwrap();
    assert_eq!(h.svm.get_account(&h.vault).map(|a| a.lamports).unwrap_or(0), 0);
}

```

The full validation harness was provided separately to the Riverboat team. To reproduce the validation run, execute from the repo root:

```

cargo test -p wager_program --test validation_runtime_phase2
  rbp2_04_locked_seat_risk_can_be_repriced_by_other_staked_seat -- --nocapture
cargo test -p wager_program --test validation_runtime_phase2
  rbp2_05_settled_high_index_seat_reuse_rewrites_existing_locked_payout -- --nocapture
cargo test -p wager_program --test validation_runtime_phase2
  rbp2_01_non_locked_stale_seat_can_claim_after_manual_push -- --nocapture

```

Test output:

```

running 1 test
test rbp2_04_locked_seat_risk_can_be_repriced_by_other_staked_seat ... ok

test result: ok. 1 passed

running 1 test
test rbp2_05_settled_high_index_seat_reuse_rewrites_existing_locked_payout ... ok

test result: ok. 1 passed

running 1 test
test rbp2_01_non_locked_stale_seat_can_claim_after_manual_push ... ok

test result: ok. 1 passed

```

Expected result:

- A locked seat's risk/payout data can be changed by a still-staked counterparty.
- After settlement and seat reuse, the victim's stored payout can be overwritten.
- A later claim debits the overwritten value from the vault, proving the mutation affects paid value.
- Manual Push settles the wager while only refreshing locked seats.
- A non-locked funded seat can claim stale payout data, become `Paid`, and lose the normal refund path.
- Remaining vault value is sweepable by the creator during close.

Recommendation

Make `payout_data` finalized-only and store provisional risk in a separate field. The important part is to compute and freeze the full risk set before any seat locks, then keep that risk set immutable for the rest of the wager lifecycle.

First, split the seat fields in `protocol/src/state.rs`:

```
pub struct Seat {
    pub discriminator: [u8; 8],
    pub wager: Pubkey,
    pub authority: Pubkey,
    pub index: u8,
    pub bump: u8,
    pub status: SeatStatus,
    pub prediction_data: Prediction,
    pub resolution_data: Option<Judgment>,
    pub risk_data: Option<Payout>, // provisional, written by prediction updates
    pub finalized_payout: Option<Payout>, // final, written only by settlement
    pub challenge: bool,
    pub last_change_at: i64,
}
```

Update `Seat::SPACE_ALLOCATION` for the extra `Option<Payout>` fields and migrate or version old accounts before deploying this layout change.

Second, reject settlement-finalized wagers and reject prediction changes once locking has started:

```
let wager = Wager::from_account(wager_account)?;

if wager.result.is_settled() {
    return Err(ProtocolError::WagerAlreadySettled.into());
}

if wager.locked_seats != 0 {
    return Err(ProtocolError::InvalidAccountStatus.into());
}
```

Keep the existing target-seat rule: the caller can only change their own `Staked` seat. Before any seat locks, all seats in the risk set should still be `Staked`; if prediction quorum exists, compute and store the full risk set in one shot:

```
for (i, account) in seat_accounts.iter().enumerate() {
    let mut seat = Seat::from_account(account)?;

    if seat.status != SeatStatus::Staked {
        return Err(ProtocolError::InvalidAccountStatus.into());
    }

    seat.risk_data = Some(Payout::Binary(payouts[i]));
    seat.last_change_at = clock.unix_timestamp;
    seat.save(account)?;
}
```

Third, require risk to exist before a seat can lock. This intentionally changes the current ordering: users can still submit predictions sequentially, but locking should only start after prediction quorum has computed the full risk set. That prevents a seat from locking with default zero risk and avoids asymmetric risk where one seat keeps old risk while another seat receives newly recomputed risk:

```
if wager.stake != 0 && seat.risk_data.is_none() {
    return Err(ProtocolError::InvalidAccountStatus.into());
}

seat.status = SeatStatus::Locked;
wager.set_seat_locked(seat.index);
```

Do not use `risk_data.value() == Some(0)` as the guard, because zero can be a valid computed risk in equal-belief cases. Use an explicit `Option` or a separate boolean that proves risk was computed.

Fourth, settlement should read frozen risk and write final payouts:

```
let risk = seat
  .risk_data
  .as_ref()
  .and_then(|payout| payout.value())
  .ok_or(ProtocolError::InvalidAccountStatus)?;

// after consensus:
for ((mut seat, account), payout) in seats.into_iter().zip(payouts.iter()) {
  seat.finalized_payout = Some(Payout::Binary(*payout));
  seat.save(account)?;
}
```

Fifth, `claim_payout` must only pay seats that settlement explicitly finalized:

```
if seat.status != SeatStatus::Locked {
  return Err(ProtocolError::InvalidAccountStatus.into());
}

let payout_amount = seat
  .finalized_payout
  .as_ref()
  .and_then(|payout| payout.value())
  .ok_or(ProtocolError::InvalidAccountStatus)?;
```

If the team does not split `risk_data` and `finalized_payout`, then at minimum add a boolean or enum that proves the payout was written by settlement:

```
pub enum PayoutState {
  Provisional(Payout),
  Final(Payout),
}

let payout_amount = match &seat.payout_data {
  PayoutState::Final(payout) => payout
    .value()
    .ok_or(ProgramError::InvalidArgument)?,
  PayoutState::Provisional(_) => {
    return Err(ProtocolError::InvalidAccountStatus.into());
  }
};
```

During Manual Push, only finalize seats that are actually payable:

```
for index in wager.get_locked_indices() {
  let seat_account = next_account_info(accounts_iter)?;
  let mut seat = Seat::from_account(seat_account)?;

  if seat.status != SeatStatus::Locked {
    return Err(ProtocolError::InvalidAccountStatus.into());
  }

  seat.finalized_payout = Some(Payout::Binary(wager.stake));
  seat.save(seat_account)?;
}
```

Add regression tests that prove:

- no seat can lock until the full risk set has been computed from a valid prediction quorum;
- after any seat locks, later `set_prediction_data` calls fail and cannot recompute only part of the risk set;

- `set_prediction_data` fails after `wager.result.is_settled()`;
- a settled seat's `finalized_payout` cannot change after seat close/reuse of another seat;
- `claim_payout` rejects Open, Reserved, and Staked seats that never received `finalized_payout`;
- creator-rendered claims use the same finalized-payout guard as direct seat-authority claims.

Team Response

Fixed and Validated. The Riverboat team now writes finalized payouts only as part of wager settlement, either through peer consensus resolution or Manual Push. `claim_payout` requires the wager result to be settled before transferring funds and marks the seat as Paid, so provisional risk data can no longer be claimed as if it were a finalized payout.

[M-03] Child wagers depend on closable and reinitializable parent contracts

Child wagers store only the parent contract account key. They do not store an immutable parent version, timing snapshot, or close guard proving that the parent state used at wager creation is still the same state used later for settlement.

This creates two symptoms from the same root cause:

- If the parent contract is closed, child wagers can be stranded because settlement paths can no longer deserialize the parent.
- If the parent contract is closed and recreated at the same PDA, child wagers can read the new parent timing and allow Manual Push earlier than the original parent would have allowed.

Relevant code paths:

- `protocol/src/state.rs:83` Wager stores only contract: Pubkey
- `wager_program/src/processor.rs:120` `make_wager` writes only the parent contract key into the child wager
- `contract_program/src/processor.rs:346` `close_contract`
- `contract_program/src/processor.rs:51` deterministic parent contract PDA
- `wager_program/src/processor.rs:77` wager creation checks current parent only
- `wager_program/src/processor.rs:667` peer resolution checks only `wager.contract == contract_account.key`
- `wager_program/src/processor.rs:730` peer resolution validates against the live parent contract outcomes
- `wager_program/src/processor.rs:924` child checks only `wager.contract == contract_account.key`
- `wager_program/src/processor.rs:935` Manual Push uses current parent `resolves_by`

Impact

A child wager can become unable to settle through parent-dependent peer-resolution or Manual Push paths after the parent is closed. If the child wager is already settled, `claim_payout` does not need the parent account. The child can also settle using timing from a recreated parent instead of the original one. The impact is limited to child wagers that reference the affected parent, and

the reinitialization variant needs the account to be closed and recreated.

This is not permissionless against any parent contract. The parent close/reinitialization setup requires the parent creator and the parent must be closable either through `result_set_at + RESOLUTION_WINDOW` or through `resolves_by + RESOLUTION_WINDOW`. The early reinitialization path before the original parent deadline also needs the parent resolution authority to set the parent result and the parent creator to close the parent; if those roles differ, this requires both roles or collusion. The early Manual Push path also requires an authorized child caller such as the child wager creator, wager resolution authority, or a locked-seat signer. The same-PDA reinitialization path needs the original creator/payer/headline combination and a zero-lamport account purge or equivalent reinitialization window.

This is Medium severity. It can block or change wager settlement, but it is still bounded by wager scope and lifecycle preconditions.

Proof of Concept

The following tests demonstrate both symptoms of the same parent-dependency issue. The preserved harness defines `assert_incorrect_program_id` and `assert_custom_error` so negative checks are pinned to the specific expected failure rather than any transaction failure.

```
#[test]
fn rbp2_02_closing_parent_contract_strands_live_child_wager() {
    let mut h = setup_real_contract_wager(NOW + 10_000, NOW + 1_000_000, "parent-close-strands-child");
    prepare_two_locked_seats(&mut h);

    let creator = h.creator.insecure_clone();
    let seat0 = h.seat0;
    let seat1 = h.seat1;

    set_contract_result(&mut h.svm, &creator, h.contract);
    set_clock(&mut h.svm, NOW + RESOLUTION_WINDOW + 1);
    close_contract_without_terms(&mut h.svm, &creator, h.contract);

    let err = set_resolution(&mut h, &creator, 0, resolved(1), &[seat0, seat1])
        .expect_err("child wager resolution unexpectedly succeeded after parent close");
    assert_incorrect_program_id(&err);

    set_clock(&mut h.svm, NOW + 1_000_000 + RESOLUTION_WINDOW + 1);
    let push_err = set_wager_push(&mut h, &creator, &[seat0, seat1])
        .expect_err("child wager push unexpectedly succeeded after parent close");
    assert_incorrect_program_id(&push_err);
}
```



```

#[test]
fn rbp2_06_reinitialized_parent_contract_enables_early_child_push() {
    let old_resolves_by = NOW + 1_000_000;
    let new_resolves_by = NOW + RESOLUTION_WINDOW + 100;

    let mut control = setup_real_contract_wager(NOW + 10_000, old_resolves_by, "parent-reinit-control");
    ;
    prepare_two_locked_seats(&mut control);

    let control_creator = control.creator.insecure_clone();
    let control_seat0 = control.seat0;
    let control_seat1 = control.seat1;

    set_clock(&mut control.svm, new_resolves_by + RESOLUTION_WINDOW + 1);
    assert!(new_resolves_by + RESOLUTION_WINDOW + 1 < old_resolves_by + RESOLUTION_WINDOW);

    let early_control_err = set_wager_push(&mut control, &control_creator, &[control_seat0,
        control_seat1])
        .expect_err("control child wager unexpectedly allowed push before original parent deadline");
    assert_custom_error(&early_control_err, "ChallengePeriodActive", 6);

    let mut h = setup_real_contract_wager(NOW + 10_000, old_resolves_by, "parent-reinit");
    prepare_two_locked_seats(&mut h);

    let creator = h.creator.insecure_clone();
    let seat0 = h.seat0;
    let seat1 = h.seat1;

    set_contract_result(&mut h.svm, &creator, h.contract);
    set_clock(&mut h.svm, NOW + RESOLUTION_WINDOW + 1);
    close_contract_without_terms(&mut h.svm, &creator, h.contract);

    purge_zero_lamport_account(&mut h.svm, h.contract);
    let reinit_contract = write_contract(&mut h.svm, &creator, "parent-reinit", new_resolves_by, 0);
    assert_eq!(reinit_contract, h.contract);

    set_clock(&mut h.svm, new_resolves_by + RESOLUTION_WINDOW + 1);
    set_wager_push(&mut h, &creator, &[seat0, seat1]).unwrap();

    assert!(get_wager(&h.svm, &h.wager.pubkey()).result.is_push());
}

```

The full validation harness was provided separately to the Riverboat team. To reproduce the validation run, execute from the repo root:

```

cargo test -p wager_program --test validation_runtime_phase2
    rbp2_02_closing_parent_contract_strands_live_child_wager -- --nocapture
cargo test -p wager_program --test validation_runtime_phase2
    rbp2_06_reinitialized_parent_contract_enables_early_child_push -- --nocapture

```

Test output:

```

running 1 test
test rbp2_02_closing_parent_contract_strands_live_child_wager ... ok

test result: ok. 1 passed

running 1 test
test rbp2_06_reinitialized_parent_contract_enables_early_child_push ... ok

test result: ok. 1 passed

```

Expected result:

- Parent close blocks child peer resolution and Manual Push paths.
- Same-PDA parent reinitialization changes child Manual Push timing.
- A control child using the original parent timing rejects early Manual Push at the same timestamp.

Recommendation

Snapshot the parent fields that child wagers rely on, then use the snapshot for child validation and Manual Push timing.

First, add a parent identity field to `Contract`. A monotonic version or nonce is better than relying on `created_at`, because `created_at` is only timestamp-based and is not a robust identity key:

```
pub struct Contract {
  pub discriminator: [u8; 8],
  pub version: u64,
  pub creator: Pubkey,
  // existing fields...
}
```

Then add the needed parent snapshot fields to `Wager`:

```
pub struct Wager {
  pub discriminator: [u8; 8],
  pub contract: Pubkey,
  pub contract_version: u64,
  pub contract_resolves_by: i64,
  pub contract_result_set_at: Option<i64>,
  pub contract_outcomes: OutcomeType,
  pub creator: Pubkey,
  // existing fields...
}
```

Populate those fields in `make_wager` immediately after reading the parent contract:

```
let contract = Contract::from_account(contract_account)?;

if !contract.finalized || contract.resolved {
  return Err(ProgramError::InvalidAccountData);
}

let wager = Wager {
  discriminator: WAGER_DISCRIMINATOR,
  contract: *contract_account.key,
  contract_version: contract.version,
  contract_resolves_by: contract.resolves_by,
  contract_result_set_at: contract.result_set_at,
  contract_outcomes: contract.outcomes.clone(),
  // existing fields...
};
```

In child peer resolution, use the snapshot instead of a mutable or re-created parent account. This path only needs the parent outcome type for `dto.resolution.is_valid`, so it should not require a live `contract_account` when the snapshot is present. Remove `contract_account` from the required accounts, or treat it as optional debug consistency only.

```
// Load only the child wager.
let mut wager = Wager::from_account(wager_account)?;

if wager.result.is_settled() {
    return Err(ProtocolError::WagerAlreadySettled.into());
}

if !dto.resolution.is_valid(&wager.contract_outcomes) {
    return Err(ProgramError::InvalidArgument);
}
```

The old live-parent gates should not run on the snapshot-only path:

```
// Remove these requirements from set_seat_resolution when using the snapshot:
if contract_account.owner != &CONTRACT_PROGRAM_ID {
    return Err(ProgramError::IncorrectProgramId);
}

if wager.contract != *contract_account.key {
    return Err(ProgramError::InvalidArgument);
}
```

This changes closed-parent behavior from "child is stranded" to "child can still complete peer resolution using the parent fields captured at child creation." That is the intended recovery behavior if snapshots are used.

In Manual Push, the parent account keyed by `wager.contract` is required so the version-gated sync/check cannot be skipped. That sync/check is owned by the stale-parent-result-window issue. Child recovery does not trust a live parent unconditionally: when that account is closed or fails the version check, Manual Push falls back to the child snapshot for challenge-window timing:

```
let window_start = wager
    .contract_result_set_at
    .unwrap_or(wager.contract_resolves_by);

if now < window_start + RESOLUTION_WINDOW {
    return Err(ProtocolError::ChallengePeriodActive.into());
}
```

Note that `contract_result_set_at` is `None` at child creation because children can only be created while the parent is not yet resolved. The late-result window is honored once the Manual Push challenge-window fix performs or requires a version-gated parent-result sync/check that populates this child field. Do not rely on the parent account remaining live forever, and do not let Manual Push trust a re-created parent at the same PDA. This parent-dependency issue owns the snapshot fields; the stale-parent-result-window issue owns the enforced sync/check that makes late parent results visible to the child.

If the protocol wants to forbid parent closure instead, add a child-wager registry owned by the contract program and reject `close_contract` while `open_child_wager_count` > 0. Do not rely on clients voluntarily passing child wager accounts to `close_contract`.

Add regression tests for:

- parent closed before child peer resolution, where peer resolution still succeeds using the snapshot,
- parent closed before Manual Push, where Push recovery still uses the snapshot timing,
- parent re-created at the same PDA with different timing,

- child resolution using the original outcome type after parent reinitialization.

Because this fix adds fields to `Wager`, it should ship with the explicit versioning and migration pattern described in the account-layout issue rather than inserting fields into live account layouts without a version gate.

Team Response

Fixed and Validated. The Riverboat team updated Manual Push to require the parent contract account, verify it matches the wager's stored `contract`, and apply the live parent challenge window when the original parent is still valid. Closed or reinitialized parent accounts are handled as expired fallback cases instead of being trusted as fresh parent state.

[M-04] Child Manual Push ignores the parent contract's late-result challenge window

Parent contract closure and child Manual Push use different challenge-window clocks.

`close_contract` treats a late parent result as starting a fresh challenge window from `contract.result_set_at.unwrap_or(contract.resolves_by)`. Child wager Manual Push only checks `contract.resolves_by + RESOLUTION_WINDOW`. If the parent result is posted after the original `resolves_by + RESOLUTION_WINDOW`, a child wager can be manually pushed immediately even though the parent contract itself cannot yet be closed.

Relevant code paths:

- `contract_program/src/processor.rs:289 set_contract_result`
- `contract_program/src/processor.rs:332` writes `contract.result_set_at`
- `contract_program/src/processor.rs:371` parent close uses `result_set_at.unwrap_or(resolves_by)`
- `wager_program/src/processor.rs:935` child Manual Push gates only on `contract.resolves_by + RESOLUTION_WINDOW`
- `wager_program/src/processor.rs:974` Manual Push writes push payout for locked seats

Impact

A child wager that already exists before the parent is resolved can be pushed before the parent result has gone through the same challenge window required by the parent lifecycle. This can block later non-push peer resolution and settle the child under stale timing assumptions.

This is low-end Medium severity. It is a real settlement-state mismatch, but the impact is bounded to child wagers that hit this late-result timing window, and the Manual Push still requires an authorized child-wager caller.

Proof of Concept

The following LiteSVM test posts a late parent result, confirms the parent cannot yet be closed, then shows that child Manual Push still succeeds. The preserved harness defines

`assert_custom_error` so negative checks are pinned to the specific expected failure rather than any transaction failure:

```
#[test]
fn rbp2_new_late_parent_result_does_not_extend_child_manual_push_window() {
    let lock_by = NOW + 100;
    let contract_resolves_by = NOW + 1_000;
    let mut h = setup_real_contract_wager(lock_by, contract_resolves_by, "late-parent-result-push");
    prepare_two_locked_seats(&mut h);

    let creator = h.creator.insecure_clone();
    let taker = h.taker.insecure_clone();
    let seat0 = h.seat0;
    let seat1 = h.seat1;

    let late_result_time = contract_resolves_by + RESOLUTION_WINDOW + 1;
    set_clock(&mut h.svm, late_result_time);
    set_contract_result(&mut h.svm, &creator, h.contract);

    let parent = Contract::deserialize(&mut &h.svm.get_account(&h.contract).unwrap().data[..]).unwrap();
    ;
    assert_eq!(parent.result_set_at, Some(late_result_time));
    assert!(late_result_time < parent.result_set_at.unwrap() + RESOLUTION_WINDOW);

    let close_err = send(
        &mut h.svm,
        &creator,
        &[],
        contract_ix(
            ContractInstruction::CloseContract,
            vec![
                AccountMeta::new(creator.pubkey(), true),
                AccountMeta::new(h.contract, false),
            ],
        ),
    ),
    )
    .expect_err("parent close unexpectedly ignored late result_set_at challenge window");
    assert_custom_error(&close_err, "ChallengePeriodActive", 6);

    set_wager_push(&mut h, &taker, &[seat0, seat1]).unwrap();
    let wager = get_wager(&h.svm, &h.wager.pubkey());
    assert!(wager.result.is_push());
    assert_eq!(payout(&h.svm, &seat0), STAKE);
    assert_eq!(payout(&h.svm, &seat1), STAKE);

    let peer_resolution = set_resolution(&mut h, &creator, 0, resolved(1), &[seat0, seat1])
        .expect_err("peer resolution unexpectedly overwrote Manual Push settlement");
    assert_custom_error(&peer_resolution, "WagerAlreadySettled", 4);
}
```

The full validation harness was provided separately to the Riverboat team. To reproduce the validation run, execute from the repo root:

```
cargo test -p wager_program --test validation_runtime_phase2
    rbp2_new_late_parent_result_does_not_extend_child_manual_push_window -- --nocapture
```

Test output:

```
running 1 test
test rbp2_new_late_parent_result_does_not_extend_child_manual_push_window ... ok

test result: ok. 1 passed
```

Expected result:

- A real parent result is posted after the original `resolves_by` + `RESOLUTION_WINDOW`.
- `close_contract` rejects at that timestamp because `result_set_at` + `RESOLUTION_WINDOW` has not elapsed.
- Child Manual Push succeeds immediately because it only uses `resolves_by` + `RESOLUTION_WINDOW`.
- The child wager is pushed and later peer resolution is blocked.

Recommendation

Use the same parent challenge-window start in child Manual Push that `close_contract` uses.

As a standalone patch, if the parent-snapshot fix from the parent-dependency issue is not adopted, `set_wager_result` can keep reading the live parent and use `contract.result_set_at.unwrap_or(contract.resolves_by)`:

```
let contract = Contract::from_account(contract_account)?;

if wager.contract != *contract_account.key {
    return Err(ProgramError::InvalidArgument);
}

let window_start = contract
    .result_set_at
    .unwrap_or(contract.resolves_by);

if now < window_start + RESOLUTION_WINDOW {
    return Err(ProtocolError::ChallengePeriodActive.into());
}
```

Do not use this older check:

```
if now < contract.resolves_by + RESOLUTION_WINDOW {
    return Err(ProtocolError::ChallengePeriodActive.into());
}
```

If the parent-snapshot fix from the parent-dependency issue is implemented, do not use a bare live-parent read as the final Manual Push dependency. Instead, Manual Push must enforce a version-gated parent-result sync/check before applying the snapshot timing rule:

1. Store the parent identity and timing snapshot described in the parent-dependency issue, including `contract_version`, `contract_resolves_by`, and `contract_result_set_at`.
2. When Manual Push is called, require the account keyed by `wager.contract` so a caller cannot omit a live parent. If that account is still owned by the contract program and has data, deserialize it, verify it is the same original parent version, and copy `contract.result_set_at` into `wager.contract_result_set_at` when a parent result exists.
3. If the original parent has been closed, or the account at that PDA no longer matches the stored parent version, do not trust any live parent fields. Fall back to the child snapshot.
4. Then compute the Manual Push window only from the child state:

```
let window_start = wager
    .contract_result_set_at
    .unwrap_or(wager.contract_resolves_by);

if now < window_start + RESOLUTION_WINDOW {
    return Err(ProtocolError::ChallengePeriodActive.into());
}
```

Do not implement this as an optional sync instruction that Manual Push can skip. A locked-seat signer can otherwise skip the sync and reproduce the PoC during the late-result challenge window. A separate sync instruction is fine as an optimization or recovery path, but `set_wager_result` must either perform the version-gated sync/check itself or require proof that the same check has already populated `wager.contract_result_set_at`.

Add regression tests for three timestamps:

```
// Before original resolves_by + window: must reject.  
// After original resolves_by + window but before result_set_at + window: must reject.  
// After result_set_at + window: may allow Manual Push if no peer consensus has settled the wager.
```

Also add regression coverage for the parent-dependency interaction:

```
// A re-created parent at the same PDA with different timing must not change the child window.  
// A live original parent with a late result must update the child window before Push can proceed.  
// A closed original parent after its late-result challenge window may fall back to the child snapshot.
```

Team Response

Fixed and Validated. The latest design removes contract-level `result` and `result_set_at` state, so Manual Push no longer has a parent-result timestamp that can be bypassed or skipped. For live parent contracts, Manual Push enforces `contract.resolves_by + RESOLUTION_WINDOW`; for closed or reinitialized parents, it falls back to the child wager's Push path.

Low Severity

[L-01] `set_contract_result` Allows Resolving Unfinalized Contracts

The `set_contract_result` instruction does not verify that a contract has been finalized before allowing the resolution authority to set its result.

Contracts that are created with `terms_size > 0` begin in an unfinalized state and are expected to be completed through `append_terms`. However, `set_contract_result` only checks signer authority, whether the result field is present, whether the result is already settled, and whether the submitted judgment is valid for the contract's outcome type. It does not enforce `contract.finalized == true`.

As a result, a resolution authority can resolve a contract while its terms are still incomplete. Afterward, the contract creator can still continue appending or finalizing terms, because `append_terms` blocks only finalized contracts and does not reject already resolved ones.

This allows the contract lifecycle to reach an inconsistent state where:

- `resolved == true`
- `finalized == false`

and terms may still be modified after resolution.

Impact

This issue does not appear to enable direct fund theft, and it does not allow wagers to be created on such contracts, since wager creation already rejects contracts that are either unfinalized or already resolved.

The impact is instead on protocol integrity and state consistency. A contract may be resolved before its terms are complete, and the terms may then be modified or finalized after the result is known. This weakens the expected lifecycle guarantees and can create misleading historical state for users, frontends, indexers, or reviewers who assume that resolution occurred only after the contract terms were finalized.

Proof of Concept

Path: `contract_program/tests/test_set_result_unfinalized.rs`

```
use borsh::{to_vec, BorshDeserialize};

use contract_program::dto::{AppendTermsData, WriteContractData};
use contract_program::instruction::ContractInstruction;
use litesvm::LiteSVM;
use protocol::ids::CONTRACT_PROGRAM_ID;
use protocol::state::{Content, Contract, EventResult, Judgment, OutcomeType, Terms};
use solana_address::Address;
use solana_program::hash::hash;
use solana_sdk::{
    instruction::{AccountMeta, Instruction},
    message::Message,
    signature::{Keypair, Signer},
    transaction::Transaction,
};
use solana_sdk_ids::system_program;
use solana_sysvar::clock::Clock;
use std::time::{SystemTime, UNIX_EPOCH};
use test_utils::HEADLINE;

fn log_contract(step: &str, contract: &Contract) {
    println!(
        "{step}: resolved={}, finalized={}, result={:?}",
        contract.resolved, contract.finalized, contract.result
    );
}
```



```

fn log_terms(step: &str, terms: &Terms) {
    println!("{step}: terms_len={}", terms.content.as_bytes().len());
}

fn send(svm: &mut LiteSVM, tx: Transaction) {
    svm.send_transaction(tx).expect("transaction failed");
}

#[test]
fn test_set_result_allows_unfinalized_contract_then_append_terms() {
    let mut svm = LiteSVM::new();
    let program_id = CONTRACT_PROGRAM_ID;
    svm.add_program_from_file(program_id, "../target/deploy/contract_program.so")
        .unwrap();

    // Step 1: Set clock.
    let mut clock = Clock::default();
    clock.unix_timestamp = SystemTime::now()
        .duration_since(UNIX_EPOCH)
        .unwrap()
        .as_secs() as i64;
    svm.set_sysvar(&clock);

    // Step 2: Alice = creator, Bob = resolution authority.
    let alice = Keypair::new();
    let bob = Keypair::new();
    svm.airdrop(&alice.pubkey(), 1_000_000_000).unwrap();
    svm.airdrop(&bob.pubkey(), 1_000_000_000).unwrap();

    // Step 3: Create contract with terms_size > 0 (starts unfinalized).
    let headline = String::from(HEADLINE);
    let headline_hash = hash(headline.as_bytes()).to_bytes();
    let resolves_by = clock.unix_timestamp + (7 * 24 * 60 * 60);

    let write_dto = WriteContractData {
        resolution_authority: Some(bob.pubkey()),
        resolves_by,
        private: false,
        category: [0u8; 8],
        hashtag: [0u8; 16],
        outcomes: OutcomeType::Binary,
        headline: Content::Plaintext(headline),
        terms_size: 64,
    };

    let (contract_pda, _) = Address::find_program_address(
        &[b"contract", alice.pubkey().as_ref(), &headline_hash[..],
        &program_id],
    );
    let (terms_pda, _) =
        Address::find_program_address(&[b"terms", contract_pda.as_ref()], &program_id);

```

```

let write_tx = Transaction::new(
    &[&alice],
    Message::new(
        &[Instruction::new_with_bytes(
            program_id,
            &to_vec(&ContractInstruction::WriteContract { dto: write_dto }).unwrap(),
            vec![
                AccountMeta::new(alice.pubkey(), true),
                AccountMeta::new_readonly(system_program::id(), false),
                AccountMeta::new(contract_pda, false),
                AccountMeta::new(terms_pda, false),
            ],
        )],
        Some(&alice.pubkey()),
    ),
    svm.latest_blockhash(),
);
send(&mut svm, write_tx);

let contract_before = Contract::deserialize(
    &mut &svm.get_account(&contract_pda).unwrap().data[..],
)
.unwrap();
assert(!(contract_before.finalized);
assert(!(contract_before.resolved);
log_contract("after create", &contract_before);
log_terms(
    "after create",
    &Terms::deserialize(&mut &svm.get_account(&terms_pda).unwrap().data[..]).unwrap(),
);

```

```

// Step 4: Bob resolves before terms are finalized.
let set_result_tx = Transaction::new(
    &[&bob],
    Message::new(
        &[Instruction::new_with_bytes(
            program_id,
            &to_vec(&ContractInstruction::SetContractResult {
                judgment: Judgment::SingleOutcome(EventResult::Resolved(1)),
            })
            .unwrap(),
            vec![
                AccountMeta::new_readonly(bob.pubkey(), true),
                AccountMeta::new(contract_pda, false),
            ],
        )],
        Some(&bob.pubkey()),
    ),
    svm.latest_blockhash(),
);
send(&mut svm, set_result_tx);

let contract_after_result = Contract::deserialize(
    &mut &svm.get_account(&contract_pda).unwrap().data[..],
)
.unwrap();
assert!(contract_after_result.resolved);
assert(!(contract_after_result.finalized);
log_contract("after set_result (bug)", &contract_after_result);

```

```

// Step 5: Alice appends terms after resolution.
let append_tx = Transaction::new(
    &[&alice],
    Message::new(
        &[Instruction::new_with_bytes(
            program_id,
            &to_vec(&ContractInstruction::AppendTerms {
                dto: AppendTermsData {
                    chunk: b"terms written after resolution".to_vec(),
                    overwrite: false,
                    finalize: false,
                },
            })
            .unwrap(),
            vec![
                AccountMeta::new(alice.pubkey(), true),
                AccountMeta::new(contract_pda, false),
                AccountMeta::new(terms_pda, false),
            ],
            Some(&alice.pubkey()),
        ),
        svm.latest_blockhash(),
    );
send(&mut svm, append_tx);

```

```

let terms_after_append = Terms::deserialize(
    &mut &svm.get_account(&terms_pda).unwrap().data[..],
)
.unwrap();
assert!(terms_after_append.content.as_bytes().len() > 0);
log_terms("after append (bug)", &terms_after_append);

```

```

// Step 6: Alice finalizes after resolution.
let finalize_tx = Transaction::new(
    &[&alice],
    Message::new(
        &[Instruction::new_with_bytes(
            program_id,
            &to_vec(&ContractInstruction::AppendTerms {
                dto: AppendTermsData {
                    chunk: b" + finalize".to_vec(),
                    overwrite: false,
                    finalize: true,
                },
            })
            .unwrap(),
            vec![
                AccountMeta::new(alice.pubkey(), true),
                AccountMeta::new(contract_pda, false),
                AccountMeta::new(terms_pda, false),
            ],
            Some(&alice.pubkey()),
        ),
        svm.latest_blockhash(),
    );

```

```

    );
    send(&mut svm, finalize_tx);

    let contract_after_finalize = Contract::deserialize(
        &mut &svm.get_account(&contract_pda).unwrap().data[..],
    )
    .unwrap();
    assert!(contract_after_finalize.resolved);
    assert!(contract_after_finalize.finalized);
    log_contract("after finalize (bug)", &contract_after_finalize);
    log_terms(
        "after finalize (bug)",
        &Terms::deserialize(&mut &svm.get_account(&terms_pda).unwrap().data[..]).unwrap(),
    );
}

```

Run the PoC - `cargo test-sbf -p contract_program --test test_set_result_unfinalized -- --nocapture test_set_result_allows_unfinalized_contract_then_append_terms`

Recommendation

Require contracts to be finalized before they can be resolved.

For example:

```

if !contract.finalized {
    return Err(ProgramError::InvalidAccountData);
}

```

Team Response

Fixed and Validated. The Riverboat team removed the contract-level result-setting path and the judgment fields from `Contract` in the latest MVP design. As a result, an unfinalized contract can no longer be marked resolved through `set_contract_result`.

[L-02] `close_contract` Can Leave an Existing Terms Account Stranded

The `close_contract` instruction treats the terms account as optional. If the caller omits the terms PDA while closing a contract that previously created terms, the program still proceeds to drain and zero the parent contract account.

Because the terms account is not required in this path, an existing program-owned terms account can remain funded after the parent contract is closed. The terms account is no longer reachable through a normal lifecycle close flow, which leaves the account effectively stranded on-chain.

The terms PDA is only closed when explicitly passed. There is no check on `contract.has_terms` before proceeding with parent closure.

Impact

This is primarily a state cleanup and rent-recovery issue.

The impact appears limited to the creator's own accounts and does not show a direct path to stealing funds from other users.

Proof of Concept

PoC file: `contract_program/tests/test_close_contract_orphan_terms.rs`

Run command

```
cargo build-sbf
cargo test-sbf -p contract_program --test test_close_contract_orphan_terms -- --nocapture
```

Complete PoC code

```
use borsh::{to_vec, BorshDeserialize};

use contract_program::dto::WriteContractData;
use contract_program::instruction::ContractInstruction;
use litesvm::LiteSVM;
use protocol::constants::RESOLUTION_WINDOW;
use protocol::ids::CONTRACT_PROGRAM_ID;
use protocol::state::{Content, Contract, OutcomeType};
use solana_address::Address;
use solana_program::hash::hash;
use solana_sdk::{
    instruction::{AccountMeta, Instruction},
    message::Message,
    signature::{Keypair, Signer},
    transaction::Transaction,
};
use solana_sdk_ids::system_program;
use solana_sysvar::clock::Clock;
use std::time::{SystemTime, UNIX_EPOCH};
use test_utils::HEADLINE;

fn send_ok(svm: &mut LiteSVM, step: &str, tx: Transaction) {
    let meta = svm
        .send_transaction(tx)
        .expect("transaction should succeed");
    println!("[{step}] tx logs: {:?}", meta.logs);
}
```

```

#[test]
fn test_close_contract_without_terms_orphans_terms_account() {
    println!("=== PoC: close_contract can orphan terms account ===");

    // Step 1: Load program and set clock.
    let mut svm = LiteSVM::new();
    let program_id = CONTRACT_PROGRAM_ID;
    svm.add_program_from_file(program_id, "../target/deploy/contract_program.so")
        .unwrap();

    let mut clock = Clock::default();
    clock.unix_timestamp = SystemTime::now()
        .duration_since(UNIX_EPOCH)
        .unwrap()
        .as_secs() as i64;
    svm.set_sysvar(&clock);
    println!("Step 1: program loaded, clock set to {}", clock.unix_timestamp);

    // Step 2: Fund creator and derive contract + terms PDAs.
    let creator = Keypair::new();
    svm.airdrop(&creator.pubkey(), 1_000_000_000).unwrap();

    let headline = String::from(HEADLINE);
    let headline_hash = hash(headline.as_bytes()).to_bytes();
    let resolves_by = clock.unix_timestamp + (7 * 24 * 60 * 60);

    let (contract_pda, _) = Address::find_program_address(
        &[b"contract", creator.pubkey().as_ref(), &headline_hash[..]],
        &program_id,
    );
    let (terms_pda, _) =
        Address::find_program_address(&[b"terms", contract_pda.as_ref()], &program_id);
    println!("Step 2: creator={}", creator.pubkey());
    println!("      contract_pda={}", contract_pda);
    println!("      terms_pda={}", terms_pda);
    println!("      resolves_by={}", resolves_by);

    // Step 3: Create contract with terms_size > 0 so has_terms=true and terms PDA is initialized.
    println!("Step 3: creating contract with terms (terms_size=64)...");
    let write_dto = WriteContractData {
        resolution_authority: Some(creator.pubkey()),
        resolves_by,
        private: false,
        category: [0u8; 8],
        hashtag: [0u8; 16],
        outcomes: OutcomeType::Binary,
        headline: Content::Plaintext(headline),
        terms_size: 64,
    };
}

```

```

let write_tx = Transaction::new(
    &[&creator],
    Message::new(
        &[Instruction::new_with_bytes(
            program_id,
            &to_vec(&ContractInstruction::WriteContract { dto: write_dto }).unwrap(),
            vec![
                AccountMeta::new(creator.pubkey(), true),
                AccountMeta::new_readonly(system_program::id(), false),
                AccountMeta::new(contract_pda, false),
                AccountMeta::new(terms_pda, false),
            ],
        )],
        Some(&creator.pubkey()),
    ),
    svm.latest_blockhash(),
);
send_ok(&mut svm, "Step 3", write_tx);

let contract_before_close =
    Contract::deserialize(&mut &svm.get_account(&contract_pda).unwrap().data[..]).unwrap();
println!(
    "Step 3 result: has_terms={}, finalized={}, resolved={}, terms_exists={}",
    contract_before_close.has_terms,
    contract_before_close.finalized,
    contract_before_close.resolved,
    svm.get_account(&terms_pda).is_some()
);
assert!(contract_before_close.has_terms);
assert!(svm.get_account(&terms_pda).is_some());

// Step 4: Advance time past the 72h close window.
clock.unix_timestamp = resolves_by + RESOLUTION_WINDOW + 1;
svm.set_sysvar(&clock);
println!(
    "Step 4: clock advanced to {} ({}s past close window)",
    clock.unix_timestamp,
    clock.unix_timestamp - (resolves_by + RESOLUTION_WINDOW)
);

// Step 5: BUG - close contract without passing the terms PDA.
println!("Step 5: closing contract WITHOUT terms_pda in account list...");
let close_without_terms_tx = Transaction::new(
    &[&creator],
    Message::new(
        &[Instruction::new_with_bytes(
            program_id,
            &to_vec(&ContractInstruction::CloseContract).unwrap(),
            vec![
                AccountMeta::new(creator.pubkey(), true),
                AccountMeta::new(contract_pda, false),
            ],
        )],
        Some(&creator.pubkey()),
    ),
    svm.latest_blockhash(),
);
send_ok(&mut svm, "Step 5", close_without_terms_tx);

```

```
// Step 6: Verify terms account is orphaned and cannot be recovered.
println!("Step 6: verifying orphaned terms state...");
assert!(
    svm.get_account(&contract_pda).is_none(),
    "contract should be closed"
);

let orphaned_terms = svm
    .get_account(&terms_pda)
    .expect("terms account should still exist (orphaned)");
println!(
    "Step 6 result: contract_exists={}, terms_exists={}, terms_owner={}, terms_lamports={}",
    svm.get_account(&contract_pda).is_some(),
    svm.get_account(&terms_pda).is_some(),
    orphaned_terms.owner,
    orphaned_terms.lamports
);
assert_eq!(orphaned_terms.owner, program_id);
assert!(
    orphaned_terms.lamports > 0,
    "orphaned terms still holds rent lamports"
);

println!("=== PoC completed: orphaned terms confirmed ===");
}
```

Recommendation

If `contract.has_terms` is true, require the terms PDA to be passed into `close_contract` and close it before draining the parent contract.

Team Response

Fixed and Validated. The Riverboat team updated `close_contract` so contracts with `has_terms` require the expected Terms account, verify its owner and PDA derivation, and close the Terms account before closing the Contract account. This prevents a valid Terms PDA from being stranded behind a closed parent contract.

[L-03] Incomplete Native Account Close Sequence Leaves Closed PDAs Program-Owned and Allocated

The close handlers manually drain lamports and zero account data, but they do not complete Solana's native account close sequence by reassigning the closed account to the System Program and reducing its data length to zero.

Affected paths:

- `contract_program/src/processor.rs::close_contract`
- `wager_program/src/processor.rs::close_wager`
- `wager_program/src/processor.rs::close_seat`

Each path only performs partial close behavior:

- drains lamports from the account
- fills account data with zeroes

However, the closed accounts remain owned by the program and retain their original data allocation until runtime cleanup. If a closed account is re-funded later in the same transaction, it may survive as a zeroed, program-owned, allocated account instead of being fully reclaimed as a clean system-owned account.

The root cause is duplicated manual close logic instead of a shared close helper that consistently applies the full native close sequence.

Impact

The immediate security impact is limited in the current implementation. The affected account types use discriminator checks, so zeroed accounts fail deserialization through `Contract::from_account`, `Wager::from_account`, and `Seat::from_account`. This prevents straightforward reuse of stale closed state.

However, the behavior remains undesirable because closed accounts may persist as program-owned zeroed accounts if re-funded in the same transaction, future instructions or migrations may accidentally treat them as valid program-owned accounts, and PDA reinitialization flows can fail when the account is not returned to a clean `system_program` owner with zero data length.

Recommendation

Centralize close behavior in a shared helper and use it for every account close path. Then replace the duplicated manual drain-and-zero logic in `close_contract`, `close_wager`, and `close_seat` with this helper.

```
pub fn close_account(
    account_to_close: &AccountInfo,
    recipient: &AccountInfo,
) -> ProgramResult {
    let close_lamports = account_to_close.lamports();

    **recipient.lamports.borrow_mut() = recipient
        .lamports()
        .checked_add(close_lamports)
        .ok_or(ProgramError::ArithmeticOverflow)?;

    **account_to_close.lamports.borrow_mut() = 0;

    account_to_close.data.borrow_mut().fill(0);
    account_to_close.assign(&system_program::ID);
    account_to_close.resize(0)?;

    Ok(())
}
```

Team Response

Fixed and Validated. The Riverboat team centralized native account closing through a shared `close_account` helper that drains lamports, zeroes account data, assigns the account back to the system program, and resizes the data length to zero. Closed PDAs are therefore no longer left program-owned and allocated.

[L-04] locked_seats bitmap is not cleared when locked seats become paid or closed

The wager program tracks locked seats using the `wager.locked_seats` bitmap.

A seat is added to the bitmap in `lock_seat`:

```
wager.set_seat_locked(seat.index);
```

The bitmap is then used by resolution-related flows such as:

```
wager.get_locked_indices()
```

However, the bitmap is only cleared in `open_seat`, and only when a `Locked` seat is reopened:

```
if seat.status == SeatStatus::Locked {  
    wager.clear_locked_seat(seat.index);  
}
```

The bitmap is **not cleared** when:

- a locked seat receives payout and becomes `Paid`,
- a paid seat is closed,
- a zero-stake locked seat is closed,
- the wager has already been settled.

This means `locked_seats` can continue to mark a seat as locked even after the actual seat status is no longer `Locked`.

Root cause: the program stores the same state in two places:

```
Seat.status  
Wager.locked_seats bitmap
```

but not every transition keeps both values synchronized.

Impact

A stale `locked_seats` bitmap can cause the wager account to report old locked seats even after those seats are paid or closed. Any instruction, client, indexer, or future protocol logic that relies on `locked_seats` as the source of truth may make incorrect decisions.

Affected areas:

- `set_seat_resolution` uses `wager.get_locked_indices()` and then expects all listed seats to still have `SeatStatus::Locked`.
- `set_wager_result` loops over `wager.get_locked_indices()` and updates payout data for those seats.
- `open_seat` uses `any_other_seat_locked()` to decide whether a locked seat can be reopened.

Proof of Concept

Alice and Bob join a wager.

1. Alice takes seat 0.
2. Bob takes seat 1.
3. Both deposit, set predictions, and lock their seats.

The wager state becomes:

```
seat 0 status = Locked
seat 1 status = Locked
locked_seats = 0b11
```

4. The wager resolves.
5. Alice claims payout.

Alice's seat status changes:

```
seat 0 status = Paid
```

But the bitmap is unchanged:

```
locked_seats = 0b11
```

Now the wager still reports Alice's seat as locked, even though the actual seat status is Paid.

6. Alice closes her paid seat.

The seat account is drained/closed, but `locked_seats` is still unchanged:

```
seat 0 = closed
seat 1 = Locked or Paid
locked_seats = 0b11
```

The bitmap now points to a seat that is no longer actually locked and may no longer even contain valid seat state.

This creates stale wager-level state that can confuse later lifecycle logic or off-chain clients.

Run the PoC test with output - `cargo test -p wager_program test_stale_locked_seats_bitmap -- --nocapture`

Test file: `wager_program/tests/test_stale_locked_seats_bitmap.rs`

Test output

```

running 1 test
[PRE-RESOLUTION] locked_seats = 0b11 (expected 0b11, correct)
[AFTER claim_payout] seat0.status = Paid (actual on-chain)
[AFTER claim_payout] wager.locked_seats = 0b11 (bit 0 still set = BUG)
[AFTER close_seat] seat0_pda lamports = 0 (0 means closed)
[AFTER close_seat] wager.locked_seats = 0b11 (bit 0 still set = BUG)
[AFTER close_seat] wager.get_locked_indices() = [0, 1]

=== PoC summary ===
After claim_payout : seat0.status=Paid but locked_seats bit 0 is STILL SET
After close_seat : seat0 account is CLOSED but locked_seats bit 0 is STILL SET
get_locked_indices() = [0, 1] - index 0 points to a closed account
=====
test test_stale_locked_seats_bitmap_after_claim_payout ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out

```

Test code

```

// test_stale_locked_seats_bitmap.rs
//
// PoC: locked_seats bitmap is not cleared when locked seats become Paid or Closed
//
// The wager program tracks which seats are locked in `wager.locked_seats` (a u64 bitmap).
// Bits are set in `lock_seat` and cleared in `open_seat` only.
// They are NEVER cleared in `claim_payout` or `close_seat`.
//
// This test proves the bitmap diverges from the actual seat status after payout
// and remains stale after the seat account is closed.

use borsh::{BorshDeserialize, to_vec};

use protocol::ids::{CONTRACT_PROGRAM_ID, SEAT_WAGER_PROGRAM_ID};
use protocol::discriminator::CONTRACT_DISCRIMINATOR;
use protocol::constants::RESOLUTION_WINDOW;
use protocol::state::{
    Contract,
    Content,
    OutcomeType,
    Wager,
    Seat,
    SeatStatus,
    Currency,
    Prediction,
    ResolutionMechanism,
    Judgment,
    EventResult,
};

```

```

use wager_program::instruction::WagerInstruction;
use wager_program::dto::{
    MakeWagerData,
    TakeSeatData,
    SeatDepositData,
    SeatPredictionData,
    SeatResolutionData,
};

use solana_address::Address;
use solana_sdk_ids::system_program;
use solana_sdk::{
    instruction::{AccountMeta, Instruction},
    message::Message,
    signature::{Keypair, Signer},
    transaction::Transaction,
    account::Account,
};

use solana_sysvar::clock::Clock;
use std::time::{SystemTime, UNIX_EPOCH};

use litesvm::LiteSVM;

fn send(svm: &mut LiteSVM, ix: Instruction, signers: &[Keypair], payer: &Keypair) {
    let msg = Message::new(&[ix], Some(&payer.pubkey()));
    let tx = Transaction::new(signers, msg, svm.latest_blockhash());
    let result = svm.send_transaction(tx);
    assert!(
        result.is_ok(),
        "transaction failed: {:?}",
        result.err()
    );
}

#[test]
fn test_stale_locked_seats_bitmap_after_claim_payout() {
    // -----
    // Setup
    // -----
    let mut svm = LiteSVM::new();
    let program_id = SEAT_WAGER_PROGRAM_ID;
    svm.add_program_from_file(program_id, "../target/deploy/wager_program.so")
        .unwrap();

    let mut clock = Clock::default();
    clock.unix_timestamp = SystemTime::now()
        .duration_since(UNIX_EPOCH)
        .unwrap()
        .as_secs() as i64;
    svm.set_sysvar(&clock);

```

```

let payer      = Keypair::new(); // seat 0 authority / wager creator
let challenger = Keypair::new(); // seat 1 authority
svm.airdrop(&payer.pubkey(),      1_000_000_000).unwrap();
svm.airdrop(&challenger.pubkey(), 1_000_000_000).unwrap();

let contract_account = Keypair::new();
let wager_account    = Keypair::new();

let stake_amount: u64 = 100_000;
let resolves_by = i64::MAX;

// -----
// 1. Mock contract
// -----
let contract_data = Contract {
    discriminator: CONTRACT_DISCRIMINATOR,
    creator: payer.pubkey(),
    resolution_authority: payer.pubkey(),
    created_at: 0,
    resolves_by,
    private: false,
    finalized: true,
    resolved: false,
    has_terms: false,
    terms_bump: 255u8,
    category: [0u8; 8],
    hashtag: [0u8; 16],
    outcomes: OutcomeType::Binary,
    headline: Content::Plaintext("PoC: stale locked_seats".to_string()),
    result: Some(Judgment::SingleOutcome(EventResult::Pending)),

    result_set_at: None,
};
let encoded = to_vec(&contract_data).unwrap();
svm.set_account(
    contract_account.pubkey(),
    Account {
        lamports: 1_000_000_000,
        data: encoded,
        owner: CONTRACT_PROGRAM_ID,
        executable: false,
        rent_epoch: 0,
    }
    .into(),
).unwrap();

// -----
// 2. make_wager
// -----
let (vault_pda, _) = Address::find_program_address(
    &[b"vault", &wager_account.pubkey().to_bytes()],
    &program_id,
);

```

```

let dto = MakeWagerData {
    seat_capacity: 2,
    stake: stake_amount,
    currency: Currency::Lamports,
    lock_by: clock.unix_timestamp + 86_400,
    private: false,
    resolution_mechanism: ResolutionMechanism::PeerConsensus,
    resolution_authority: None,
};
send(
    &mut svm,
    Instruction::new_with_bytes(
        program_id,
        &to_vec(&WagerInstruction::MakeWager { dto }).unwrap(),
        vec![
            AccountMeta::new(payer.pubkey(), true),
            AccountMeta::new_readonly(system_program::id(), false),
            AccountMeta::new(contract_account.pubkey(), false),
            AccountMeta::new(wager_account.pubkey(), true),
            AccountMeta::new(vault_pda, false),
        ],
    ),
    &[&payer, &wager_account],
    &payer,
);

```

```

// -----
// 3. take_seat - payer (index 0) and challenger (index 1)
// -----
let raw_wager = svm.get_account(&wager_account.pubkey()).unwrap();
let w = Wager::deserialize(&mut &raw_wager.data[..]).unwrap();

let (seat0_pda, _) = Address::find_program_address(
    &[b"seat", &wager_account.pubkey().to_bytes(), &[w.seat_count]],
    &program_id,
);

send(
    &mut svm,
    Instruction::new_with_bytes(
        program_id,
        &to_vec(&WagerInstruction::TakeSeat {
            dto: TakeSeatData { authority: payer.pubkey() },
        }).unwrap(),
        vec![
            AccountMeta::new(payer.pubkey(), true),
            AccountMeta::new_readonly(system_program::id(), false),
            AccountMeta::new(contract_account.pubkey(), false),
            AccountMeta::new(wager_account.pubkey(), false),
            AccountMeta::new(seat0_pda, false),
        ],
    ),
    &[&payer],
    &payer,
);

```

```

let raw_wager = svm.get_account(&wager_account.pubkey()).unwrap();
let w = Wager::deserialize(&mut &raw_wager.data[..]).unwrap();

let (seat1_pda, _) = Address::find_program_address(
    &[b"seat", &wager_account.pubkey().to_bytes(), &[w.seat_count]],
    &program_id,
);

send(
    &mut svm,
    Instruction::new_with_bytes(
        program_id,
        &to_vec(&WagerInstruction::TakeSeat {
            dto: TakeSeatData { authority: challenger.pubkey() },
        }).unwrap(),
        vec![
            AccountMeta::new(challenger.pubkey(), true),
            AccountMeta::new_readonly(system_program::id(), false),
            AccountMeta::new(contract_account.pubkey(), false),
            AccountMeta::new(wager_account.pubkey(), false),
            AccountMeta::new(seat1_pda, false),
        ],
    ),
    &[&challenger],
    &challenger,
);

// -----
// 4. deposit - both seats
// -----
for (user, seat_pda) in [(&payer, seat0_pda), (&challenger, seat1_pda)] {
    send(
        &mut svm,
        Instruction::new_with_bytes(
            program_id,
            &to_vec(&WagerInstruction::ProcessDeposit {
                dto: SeatDepositData { amount: stake_amount },
            }).unwrap(),
            vec![
                AccountMeta::new(user.pubkey(), true),
                AccountMeta::new_readonly(system_program::id(), false),
                AccountMeta::new(wager_account.pubkey(), false),
                AccountMeta::new(vault_pda, false),
                AccountMeta::new(seat_pda, false),
            ],
        ),
        &[user],
        user,
    );
}

```



```

// -----
// 5. set_prediction_data - payer 95, challenger 13
// -----
for (user, idx, prediction) in [
  (&payer,      0u8, Prediction::Binary(95u8)),
  (&challenger, 1u8, Prediction::Binary(13u8)),
] {
  send(
    &mut svm,
    Instruction::new_with_bytes(
      program_id,
      &to_vec(&WagerInstruction::SetPredictionData {
        dto: SeatPredictionData { seat_index: idx, prediction },
      }).unwrap(),
      vec![
        AccountMeta::new(user.pubkey(), true),
        AccountMeta::new(wager_account.pubkey(), false),
        AccountMeta::new(seat0_pda, false),
        AccountMeta::new(seat1_pda, false),
      ],
    ),
    &[user],
    user,
  );
}

```

```

// -----
// 6. lock_seat - both seats
// -----
for (user, seat_pda) in [&payer, seat0_pda], [&challenger, seat1_pda] {
  send(
    &mut svm,
    Instruction::new_with_bytes(
      program_id,
      &to_vec(&WagerInstruction::LockSeat).unwrap(),
      vec![
        AccountMeta::new(user.pubkey(), true),
        AccountMeta::new(wager_account.pubkey(), false),
        AccountMeta::new(seat_pda, false),
      ],
    ),
    &[user],
    user,
  );
}

```

```

// --- Verify: both bits are set BEFORE resolution ---
let raw_wager = svm.get_account(&wager_account.pubkey()).unwrap();
let wager_before = Wager::deserialize(&mut &raw_wager.data[..]).unwrap();
assert!(
    wager_before.is_seat_locked(0),
    "pre-condition: seat 0 should be marked locked before resolution"
);
assert!(
    wager_before.is_seat_locked(1),
    "pre-condition: seat 1 should be marked locked before resolution"
);
assert_eq!(wager_before.locked_count(), 2, "pre-condition: locked_count should be 2");
println!(
    "[PRE-RESOLUTION] locked_seats = 0b{:02b} (expected 0b11, correct)",
    wager_before.locked_seats
);

// -----
// 7. set_seat_resolution - both agree on Outcome 1 -> consensus
// -----
for (user, idx) in [&payer, 0u8], [&challenger, 1u8] {
    send(
        &mut svm,
        Instruction::new_with_bytes(
            program_id,
            &to_vec(&WagerInstruction::SetSeatResolution {
                dto: SeatResolutionData {
                    seat_index: idx,
                    resolution: Judgment::SingleOutcome(EventResult::Resolved(1)),
                },
            },
        ),
        &[user],
        user,
    );
}

// -----
// 8. claim_payout - payer (seat 0) collects winnings
// -----
send(
    &mut svm,
    Instruction::new_with_bytes(
        program_id,
        &to_vec(&WagerInstruction::ClaimPayout).unwrap(),
        vec![
            AccountMeta::new(payer.pubkey(), true),
            AccountMeta::new_readonly(system_program::id(), false),
            AccountMeta::new(wager_account.pubkey(), false),
            AccountMeta::new(vault_pda, false),
            AccountMeta::new(seat0_pda, false),
            AccountMeta::new(payer.pubkey(), false), // seat_authority
        ],
    ),
    &[payer],
    payer,
);

```

```

    ),
    &[&payer],
    &payer,
);

// -----
// BUG ASSERTION 1: bitmap not cleared after claim_payout
// -----
let raw_wager = svm.get_account(&wager_account.pubkey()).unwrap();
let wager_after_payout = Wager::deserialize(&mut &raw_wager.data[..]).unwrap();

let raw_seat0 = svm.get_account(&seat0_pda).unwrap();
let seat0_after_payout = Seat::deserialize(&mut &raw_seat0.data[..]).unwrap();

println!(
    "[AFTER claim_payout] seat0.status = {:?}    (actual on-chain)",
    seat0_after_payout.status
);
println!(
    "[AFTER claim_payout] wager.locked_seats = 0b{:02b}    (bit 0 still set = BUG)",
    wager_after_payout.locked_seats
);

// seat 0 is now Paid - the lock bit should have been cleared
assert_eq!(
    seat0_after_payout.status,
    SeatStatus::Paid,
    "seat 0 must be Paid after claim_payout"
);

// BUG: the bitmap still marks seat 0 as locked even though it is Paid
assert!(
    wager_after_payout.is_seat_locked(0),
    "BUG CONFIRMED: locked_seats bit 0 is still set after seat becomes Paid"
);

// Prove the desync: seat status says Paid, bitmap says Locked
assert_ne!(
    seat0_after_payout.status,
    SeatStatus::Locked,
    "seat 0 is NOT Locked (it's Paid)"
);
assert_eq!(
    wager_after_payout.locked_count(),
    2,
    "BUG CONFIRMED: locked_count still reports 2 even though seat 0 has been paid out"
);

// -----
// 9. close_seat - payer closes their Paid seat
// -----
// Advance clock past result_set_at so the close is allowed
let result_set_at = wager_after_payout.result_set_at.unwrap();
clock.unix_timestamp = result_set_at + RESOLUTION_WINDOW + 1;
svm.set_sysvar(&clock);

```

```

send(
    &mut svm,
    Instruction::new_with_bytes(
        program_id,
        &to_vec(&WagerInstruction::CloseSeat).unwrap(),
        vec![
            AccountMeta::new(payer.pubkey(), true),
            AccountMeta::new(wager_account.pubkey(), false),
            AccountMeta::new(seat0_pda, false),
            AccountMeta::new(payer.pubkey(), false), // seat_authority
        ],
    ),
    &[&payer],
    &payer,
);

// -----
// BUG ASSERTION 2: bitmap not cleared after close_seat
// -----
let raw_wager = svm.get_account(&wager_account.pubkey()).unwrap();
let wager_after_close = Wager::deserialize(&mut &raw_wager.data[..]).unwrap();

// seat 0 account is now closed (zero lamports)
let seat0_lamports = svm
    .get_account(&seat0_pda)
    .map(|a| a.lamports)
    .unwrap_or(0);

println!(
    "[AFTER close_seat] seat0_pda lamports = {}    (0 means closed)",
    seat0_lamports
);
println!(
    "[AFTER close_seat] wager.locked_seats = 0b{:02b}    (bit 0 still set = BUG)",
    wager_after_close.locked_seats
);
println!(
    "[AFTER close_seat] wager.get_locked_indices() = {:?}" ,
    wager_after_close.get_locked_indices()
);

// Seat account is closed
assert_eq!(seat0_lamports, 0, "seat 0 account should be closed");

// BUG: bitmap still says seat 0 is locked even though the account is gone
assert!(
    wager_after_close.is_seat_locked(0),
    "BUG CONFIRMED: locked_seats bit 0 is still set after seat is closed"
);

// get_locked_indices() now returns a stale index pointing to a closed account
let stale_indices = wager_after_close.get_locked_indices();
assert!(
    stale_indices.contains(&0),
    "BUG CONFIRMED: get_locked_indices() returns 0 - a closed account - as 'locked'"
);

println!();
println!("=== PoC summary ===");
println!("After claim_payout : seat0.status=Paid but locked_seats bit 0 is STILL SET");
println!("After close_seat : seat0 account is CLOSED but locked_seats bit 0 is STILL SET");
println!("get_locked_indices() = {:?} - index 0 points to a closed account", stale_indices);
println!("=====");
}

```

Recommendation

Keep `locked_seats` synchronized with the actual seat lifecycle.

Whenever a seat leaves the `Locked` state permanently, clear its bit.

At minimum, clear the bit in:

1. `claim_payout`, after the seat becomes `Paid`:

```
wager.clear_locked_seat(seat.index);
wager.save(wager_account)?;
seat.status = SeatStatus::Paid;
seat.save(seat_account)?;
```

This requires `claim_payout` to deserialize `wager` as mutable.

2. `close_seat`, before closing a `Locked` or `Paid` seat:

```
if wager.is_seat_locked(seat.index) {
    wager.clear_locked_seat(seat.index);
    wager.save(wager_account)?;
}
```

Team Response

Acknowledged. The Riverboat team intentionally keeps `locked_seats` as a persistent lock-history signal after resolution, rather than treating it as the current source of truth for live locked seats. The stale bitmap behavior remains present by design, so the final report should count this as acknowledged residual risk.

[L-05] Non-Canonical `seat_count` Accounting Can Desynchronize Wager State From Actual Seat PDA Indices

The `wager` program uses `wager.seat_count` as if it represents both:

1. the number of active/live seats, and
2. the next seat index to derive when creating a new seat.

This creates an accounting mismatch because seat PDAs are indexed by fixed PDA seeds:

```
[b"seat", wager_account.key.as_ref(), &index.to_le_bytes()]
```

In `take_seat`, the program derives the next fresh seat PDA from `wager.seat_count`:

```
let (index, bump) =
    verify_or_find_seat_pda(program_id, wager_account, seat_account, wager.seat_count)?;
```

Inside `verify_or_find_seat_pda`, if the seat does not already exist as an open program-owned account, the expected PDA is derived from `seat_count`:

```
let (pda, bump) = Address::find_program_address(
    &[b"seat", wager_account.key.as_ref(), &seat_count.to_le_bytes()],
    program_id,
);
```

However, `close_seat` blindly decrements `seat_count` regardless of which seat index was closed:

```
wager.seat_count -= 1;
```

This means that if seat 0 is closed while seat 1 still exists, `seat_count` becomes 1, even though the remaining live seat is index 1. The program state now implies a compact seat set [0], while the actual remaining PDA is [1].

The root cause is that `seat_count` is being treated as a compact live-seat counter, but seat PDAs are non-compact fixed-index accounts. Once individual seats can be closed independently, the invariant "live seats are always indexed from 0 to `seat_count - 1`" no longer holds.

Function-by-function impact:

- `take_seat`: affected because new seat PDA derivation depends on `seat_count`.
- `set_prediction_data`: affected in design because it expects exactly `seat_count` accounts and then uses `enumerate()` as the expected seat index.
- `close_seat`: root cause trigger; it decrements `seat_count` without considering which index was closed.

Impact

The direct impact is **state desynchronization** between the wager account and the actual seat PDA layout.

The most sensitive affected pattern is in `set_prediction_data`, which requires:

```
seat_accounts.len() == wager.seat_count
```

Then it derives expected seat PDAs using the loop index:

```
for (i, account) in seat_accounts.iter().enumerate() {
    // expects seat index i
}
```

So if only seat 1 remains but `seat_count == 1`, the function expects the passed account to be seat 0. Passing the real remaining seat 1 fails validation.

That said, in the current lifecycle, `close_seat` usually happens after active participation windows, so the practical impact is reduced.

Proof of Concept

Alice and Bob participate in a two-seat wager.

Initial state:

```
seat_count = 0
live_seats = []
```

1. Alice takes the first seat.

```
Alice seat = index 0
seat_count = 1
live_seats = [0]
```

2. Bob takes the second seat.

```
Bob seat = index 1
seat_count = 2
live_seats = [0, 1]
```

3. Alice's seat later becomes closeable, for example after the wager lifecycle reaches a state where `close_seat` permits closing.

4. Alice closes seat 0.

`close_seat` drains Alice's seat and executes:

```
wager.seat_count -= 1;
```

New state:

```
seat_count = 1
live_seats = [1]
```

This is inconsistent.

The wager account now suggests there is one live seat at index 0, but the actual remaining seat is Bob's seat at index 1.

5. A later instruction or client that relies on `seat_count` as a compact index range now assumes the live seat set is:

```
[0]
```

But the actual live seat set is:

```
[1]
```

For example, `set_prediction_data` would require one seat account because `seat_count == 1`, then derive the expected PDA using loop index 0. If Bob passes his real seat account at index 1, PDA

validation fails because the program expected seat 0.

This demonstrates that `seat_count` can become a misleading tracking value after out-of-order seat closure.

Recommendation

Do not use `seat_count` as both the live-seat count and the next PDA index.

Treat seat indices as fixed and non-compact. If `seat_count` is used as the next-created index, it should not be decremented in `close_seat`. If the program needs to track how many seats are currently live, add a separate field such as `active_seat_count`.

Team Response

Fixed and Validated. The Riverboat team removed the `wager.seat_count -= 1` mutation from `close_seat`, so closing an already-created seat no longer moves the allocation cursor backward. Seat PDA indices remain stable across cleanup, and the latest code uses capacity-indexed seat PDA checks for final wager closure.

[L-06] Incomplete seat state reset on open_seat and take_seat leaves stale resolution_data

When a seat is reopened or retaken, the program resets some fields but not all coupled seat state.

`open_seat` (`wager_program/src/processor.rs`) sets `status = Open` and resets `prediction_data` and `payout_data`, but does **not** reset:

- `resolution_data`
- `challenge`
- `last_change_at`

`take_seat` **open-seat replacement** (`wager_program/src/processor.rs`) only updates `authority`, `status`, and `last_change_at`. It does not rebuild `prediction_data`, `payout_data`, `resolution_data`, or `challenge`.

By contrast, a newly created seat initializes all fields together (`wager_program/src/processor.rs`).

`set_seat_resolution` builds consensus from every locked seat's stored `resolution_data`, not only the signer's fresh write (`wager_program/src/processor.rs`). In theory, a stale judgment on one seat could count toward `consensus::resolve_peers()` when another participant submits a matching resolution.

However, in the current MVP this is largely constrained by timing rules:

- Non-Pending `resolution_data` can only be written while a seat is `Locked`.
- A `Locked` seat can only be opened after `wager.lock_by` has passed.
- `take_seat` on the same wager fails once `clock >= wager.lock_by`.

So a seat cannot realistically be retaken on the **same wager** with stale non-Pending resolution data. Before-deadline Open seats come from Reserved/Staked opens and still have default Pending resolution.

Additionally, `challenge` is persisted but never read in any handler, and `last_change_at` is not enforced on-chain.

Impact

The practical impact today is limited to:

1. **State inconsistency** - Reopened/retaken seats can carry lifecycle data from a prior round, which is incorrect and brittle if seat-reuse rules change.
2. **Off-chain confusion** - Indexers/clients may display stale resolution or challenge state for a seat that should be fresh.

Proof of Concept

1. **Alice** takes seat 0 (`stake > 0`), deposits, then `open_seat` before locking.
2. Seat is Open; `prediction_data` and `payout_data` are reset, but `resolution_data` remains whatever was previously stored (default Pending on first cycle).
3. **Bob** calls `take_seat` on the open seat. Only `authority`, `status`, and `last_change_at` change; other fields are not rebuilt to match a fresh Seat initialization.

Recommendation

Reset all seat lifecycle fields whenever a seat is reopened and ideally when an open seat is retaken, so stale state cannot carry across occupants.

At minimum in `open_seat`, clear both the fields below alongside existing prediction/payout resets.

```
seat.resolution_data = Some(Judgment::SingleOutcome(EventResult::Pending));
seat.challenge = false;
```

Team Response

Fixed and Validated. The Riverboat team updated the seat reopening flow so `open_seat` resets `prediction_data`, `resolution_data`, `payout_data`, and `challenge` before saving the reopened seat. New or retaken seats are also initialized with default resolution and payout state, so stale participant-controlled resolution data is no longer carried into a fresh seat lifecycle.

[L-07] `close_seat` sends another participant's seat rent to the wager creator during delayed cleanup

`close_seat` allows a wager creator to close another participant's eligible seat after the post-settlement cleanup window. That cleanup authority appears intentional: the creator can eventually clean up inactive participants' seats so the wager can be finalized.

The bug is that the closed seat PDA's lamports are credited to the caller (`signer`) instead of the stored seat authority.

The instruction first verifies that the passed `seat_authority` account matches the authority stored in the seat:

```
// Verify seat_authority matches seat and passed-in account
if seat.authority != *seat_authority.key {
  return Err(ProgramError::InvalidAccountData);
}
```

It then allows the wager creator to close a Paid seat after the challenge window:

```
// Wager creator closes a Paid seat - 72hrs after result_set_at
(false, true, SeatStatus::Paid) => {
  let result_set_at = wager.result_set_at
  .ok_or(ProtocolError::InvalidAccountStatus)?;
  if now < result_set_at + RESOLUTION_WINDOW {
    return Err(ProtocolError::InvalidAccountStatus.into());
  }
}
```

However, the close recipient is `signer`:

```
close_account(seat_account, signer)?;
```

`close_account` credits all lamports in the account being closed to its recipient:

```
let close_lamports = account_to_close.lamports();

**recipient.lamports.borrow_mut() = recipient
.lamports()
.checked_add(close_lamports)
.ok_or(ProgramError::ArithmeticOverflow)?;

**account_to_close.lamports.borrow_mut() = 0;
```

As a result, if the seat authority closes their own seat, the rent returns to the seat authority. But if the wager creator waits until they are allowed to perform delayed cleanup and closes that same seat first, the seat rent is transferred to the wager creator instead.

This contradicts the cleanup comment in `close_seat`:

```
// Close a Seat and return rent to seat_authority.
```

Impact

The wager creator can capture another participant's seat account rent during delayed cleanup.

In a two-party wager:

1. Alice creates the wager.
2. Bob joins the wager and becomes the authority of his seat.
3. The wager settles and Bob's seat becomes Paid.
4. Bob can close his own seat and recover the seat PDA rent.
5. If Bob does not close it first, Alice can wait until `result_set_at + RESOLUTION_WINDOW`.
6. Alice calls `close_seat` on Bob's seat.
7. The program validates that Bob is the seat authority, but sends the closed seat lamports to Alice because Alice is the signer.

This does not steal Bob's stake or payout. It is limited to the rent held by the seat PDA, and the creator must wait until the delayed cleanup window allows creator-side closure. For that reason, Low severity is appropriate.

The issue is still security-relevant because a privileged cleanup path transfers third-party rent to the cleanup caller instead of the account that owns the seat.

Proof of Concept

The following LiteSVM test reproduces the issue. The helper functions in the test harness register the wager program, seed the wager and seat accounts, and send a signed `closeSeat` instruction.

```
#[test]
fn poc_litesvm_wager_creator_can_capture_other_user_seat_rent() {
    let _guard = TEST_LOCK
    .get_or_init(|| Mutex::new(()))
    .lock()
    .unwrap_or_else(|poisoned| poisoned.into_inner());

    let mut svm = LiteSVM::new();
    register_program(&mut svm);

    let result_set_at = 100;
    set_clock(&mut svm, result_set_at + RESOLUTION_WINDOW + 1);

    let creator = Keypair::new();
    let seat_authority = Keypair::new();
    let wager_key = Address::new_unique();
    let (vault_key, vault_bump) =
        Address::find_program_address(&[b"vault", wager_key.as_ref()], &SEAT_WAGER_PROGRAM_ID);
    let (seat_key, seat_bump) = Address::find_program_address(
        &[b"seat", wager_key.as_ref(), &[0]],
        &SEAT_WAGER_PROGRAM_ID,
    );

    let creator_start = 1_000_000_000;
    let authority_start = 1_000_000;
    let seat_rent = 50_000_000;
```

```

set_account(
    &mut svm,
    creator.pubkey(),
    system_program::ID,
    creator_start,
    Vec::new(),
);
set_account(
    &mut svm,
    seat_authority.pubkey(),
    system_program::ID,
    authority_start,
    Vec::new(),
);
set_account(
    &mut svm,
    wager_key,
    SEAT_WAGER_PROGRAM_ID,
    10_000_000,
    serialize(&build_wager(
        creator.pubkey(),
        vault_bump,
        1,
        1,
        Some(result_set_at),
    )),
);
set_account(&mut svm, vault_key, SEAT_WAGER_PROGRAM_ID, 1, Vec::new());
set_account(
    &mut svm,

```

```

    seat_key,
    SEAT_WAGER_PROGRAM_ID,
    seat_rent,
    serialize(&build_paid_seat(
        wager_key,
        seat_authority.pubkey(),
        0,
        seat_bump,
    )),
);

let instruction = Instruction::new_with_bytes(
    SEAT_WAGER_PROGRAM_ID,
    &serialize(&wager_program::instruction::WagerInstruction::CloseSeat),
    vec![
        AccountMeta::new(creator.pubkey(), true),
        AccountMeta::new(wager_key, false),
        AccountMeta::new(seat_key, false),
        AccountMeta::new_readonly(seat_authority.pubkey(), false),
    ],
);

let metadata = send_single_ix(&mut svm, &creator, instruction).unwrap();

let creator_after = svm.get_account(&creator.pubkey()).unwrap().lamports;
let authority_after = svm
    .get_account(&seat_authority.pubkey())
    .unwrap()
    .lamports;

assert!(svm.get_account(&seat_key).is_none());
assert_eq!(creator_after, creator_start + seat_rent - metadata.fee);
assert_eq!(authority_after, authority_start);
}

```

The seat PDA is closed, but the rent is credited to the creator. The seat authority's balance is

unchanged.

LiteSVM run command:

```
cargo test -p wager_program --test poc_close_regressions
    poc_litesvm_wager_creator_can_capture_other_user_seat_rent -- --nocapture
```

Test output (LiteSVM):

```
Finished `test` profile [unoptimized + debuginfo] target(s) in 0.59s
Running tests/poc_close_regressions.rs (target/debug/deps/poc_close_regressions-dc2274e7c30080f1)

running 1 test
Seat account closed successfully!
Wager.seat_count decremented to 0
test poc_litesvm_wager_creator_can_capture_other_user_seat_rent ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 1 filtered out; finished in 0.07s
```

The same behavior was also reproduced through a Surfpool RPC test by seeding the wager and seat accounts with `surfnet_setAccount`, sending a signed `CloseSeat` transaction from the creator, and observing that the creator's balance increased while the seat authority's balance did not.

Surfpool setup and run commands:

```
cargo build-sbf --tools-version v1.52
NO_DNA=1 surfpool start --offline --ci --port 18899 --ws-port 18900 --artifacts-path target/deploy --
yes
cargo test -p wager_program --test poc_close_regressions_surfpool -- --ignored --nocapture --test-
threads=1
```

Test output (Surfpool):

```
Finished `test` profile [unoptimized + debuginfo] target(s) in 0.16s
Running tests/poc_close_regressions_surfpool.rs (target/debug/deps/poc_close_regressions_surfpool-2
e5747e3e8e429aa)

running 1 test
test poc_surfpool_close_regressions ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.05s
```

Recommendation

In `wager_program/src/processor.rs`, update `close_seat`.

Replace the close recipient at line 1318:

```
close_account(seat_account, signer)?;
```

with:

```
close_account(seat_account, seat_authority)?;
```

Because `close_account` mutates the recipient lamports, the `seat_authority` account must be writable

for `closeSeat` calls. Any client or test helper that currently passes `seat_authority` as `readonly` should change that account meta to `writable`.

For example, change the `closeSeat` account metas from:

```
AccountMeta::new_readonly(seat_authority.pubkey(), false)
```

to:

```
AccountMeta::new(seat_authority.pubkey(), false)
```

The existing `seat.authority == *seat_authority.key` check should remain, so creator-side cleanup can still close inactive seats while returning rent to the participant who owned the seat.

Add regression coverage for both close paths:

- seat authority closes their own paid seat and receives rent
- wager creator closes another authority's paid seat after `RESOLUTION_WINDOW` and the seat authority, not the creator, receives rent

Team Response

Fixed and Validated. The Riverboat team updated `close_seat` to require the stored `seat_authority` account and pass that account as the close recipient. This preserves the wager creator's delayed cleanup ability while ensuring another participant's seat rent is returned to the participant's authority rather than the wager creator.

Informational Severity

[I-01] `write_contract` Allows Blank Contract Headlines

The contract creation flow validates only the maximum byte length of the submitted headline and does not require the headline to contain any non-whitespace content.

In `write_contract`, the program derives the contract PDA using `hash(dto.headline.as_bytes())`, then only rejects headlines whose byte length exceeds `MAX_HEADLINE_LEN`. Since `Content::len()` returns the raw byte length of the underlying plaintext string, both empty strings and whitespace-only strings are accepted as valid headlines.

As a result, users can create contracts with effectively blank human-readable identifiers, such as:

- `""`
- `" "`
- `"\n\t"`

This is especially relevant because contracts may be finalized with `terms_size == 0`, meaning a blank headline can leave the contract with no meaningful visible summary.

Proof of Concept

Submit `WriteContractData` with an empty plaintext headline:

```
headline: Content::Plaintext(String::new())
```

Or with a whitespace-only headline:

```
headline: Content::Plaintext("   ".to_string())
```

Both inputs pass the current validation as long as their byte length is less than or equal to `MAX_HEADLINE_LEN`.

Recommendation

Reject headlines that are empty or whitespace-only after trimming.

For example:

```
match &dto.headline {
  Content::Plaintext(s) if s.trim().is_empty() => {
    return Err(ProgramError::InvalidArgument);
  }
  _ => {}
}
```

Team Response

Fixed and Validated. The Riverboat team added validation in `write_contract` that rejects blank or whitespace-only plaintext headlines and continues to enforce headline length bounds. Contracts can no longer be created with an empty visible headline.

[I-02] `open_seat` relies on unchecked lamport arithmetic for stake refunds

In `open_seat`, the program moves stake from the vault back to the seat authority using raw lamport mutation:

```
**vault_account.try_borrow_mut_lamports()? -= stake;
**seat_authority.try_borrow_mut_lamports()? += stake;
```

Recommendation

Replace the raw lamport mutation with checked arithmetic and a clear error path, for example:

```

let stake = wager.stake;

let vault_balance = vault_account.lamports();
let authority_balance = seat_authority.lamports();

let new_vault_balance = vault_balance
    .checked_sub(stake)
    .ok_or(ProgramError::InsufficientFunds)?;

let new_authority_balance = authority_balance
    .checked_add(stake)
    .ok_or(ProgramError::ArithmeticOverflow)?;

**vault_account.try_borrow_mut_lamports()? = new_vault_balance;
**seat_authority.try_borrow_mut_lamports()? = new_authority_balance;

```

Team Response

Fixed and Validated. The Riverboat team uses checked lamport arithmetic for vault transfers and verifies the seat authority and vault PDA before refunding stake in `open_seat`. Stake refunds therefore no longer rely on unchecked lamport subtraction or addition.

[I-03] Account discriminator is checked after full Borsh deserialization

`AccountSerialize::from_account()` deserializes the entire account buffer before validating the account discriminator. For invalid or substituted accounts, the program may spend unnecessary compute parsing account data that could have been rejected by checking the first 8 bytes directly.

This is especially relevant for account types containing variable-length fields, such as `Contract.headline` and `Terms.content`.

Recommendation

Check the first 8 bytes of account data against `Self::DISCRIMINATOR` before calling `BorshDeserialize`. Only deserialize the full account after the discriminator has been validated.

The safer pattern is:

```

let data = account.try_borrow_data()?;

if data.len() < 8 || data[..8] != Self::DISCRIMINATOR {
    return Err(ProgramError::InvalidAccountData);
}

let deserialized = Self::deserialize(&mut &data[..])?;

```


Team Response

Fixed and Validated. The Riverboat team added 8-byte account discriminators and centralized account loading through discriminator-checking helpers. Account data is now type-checked before Borsh deserialization is trusted by instruction logic.

[I-04] Arbitrary seat authority can leave seat PDAs uncleared during cleanup

`take_seat` stores caller-controlled `TakeSeatData.authority` without requiring that authority to sign or proving it is a writable normal account that can later receive lamports.

This creates cleanup edge cases from the same root cause:

- A seat can set its authority to the seat PDA itself.
- A seat can set its authority to a builtin or otherwise protected recipient such as `system_program::id()`.

Normal writable wallet authorities do not create this issue. The harmful cases are authorities that cannot safely receive direct lamport credits or that alias the seat PDA itself. When cleanup later attempts to credit rent to the stored authority, the close path can fail or leave a nonzero-lamport, no-longer-deserializable seat PDA. `close_wager` then cannot complete because it requires all expected seat PDAs to have zero lamports.

Relevant code paths:

- `wager_program/src/processor.rs:257` stores arbitrary authority
- `wager_program/src/processor.rs:1241` close checks the passed authority only equals stored authority
- `wager_program/src/processor.rs:1300` close credits lamports to the authority
- `wager_program/src/processor.rs:1154` `close_wager` requires zero-lamport seat PDAs

Impact

An attacker can cause a per-wager cleanup failure in the harmful authority cases. This can leave seat rent or residual cleanup value unrecoverable through the normal close path, but it does not block ordinary payout claims and does not let the attacker steal other users' funds.

This is Informational because the impact is limited to cleanup state and small residual value in the affected wager. It is still useful to report because the close path assumes the stored authority is a safe lamport recipient, but `take_seat` does not enforce that assumption.

Proof of Concept

The following tests show two ways the arbitrary authority can interfere with cleanup: self-PDA authority and `system_program::id()` as authority. Other protected recipients such as sysvars, executable accounts, or readonly recipients should be covered by follow-up regression tests rather than treated as already proven by these artifacts.

```

#[test]
fn self_pda_authority_leaves_seat_pda_uncleared() {
    let mut h = setup_mock_wager(STAKE, NOW + 100, NOW + 1_000_000);
    let creator = h.creator.insecure_clone();
    let seat0 = h.seat0;

    take_seat(&mut h, &creator, seat0, seat0);

    set_clock(&mut h.svm, NOW + 101);
    open_seat(&mut h, &creator, seat0, seat0).unwrap();

    let close_result = close_seat(&mut h, &creator, seat0, seat0);
    let seat_account = h.svm.get_account(&seat0).unwrap();

    if close_result.is_ok() {
        assert!(seat_account.lamports > 0);
        assert!(account_data_is_zeroed(&h.svm, &seat0));
    } else {
        let stuck_seat = get_seat(&h.svm, &seat0);
        assert_eq!(stuck_seat.authority, seat0);
        assert_eq!(stuck_seat.status, SeatStatus::Open);
        assert!(seat_account.lamports > 0);
    }

    assert!(close_wager(&mut h).is_err());
}

```

```

#[test]
fn system_program_authority_prevents_seat_cleanup() {
    let mut h = setup_mock_wager(STAKE, NOW + 100, NOW + 1_000_000);
    let creator = h.creator.insecure_clone();
    let seat0 = h.seat0;

    take_seat(&mut h, &creator, seat0, system_program::id());

    set_clock(&mut h.svm, NOW + 101);
    open_seat(&mut h, &creator, seat0, system_program::id()).unwrap();

    let err = close_seat(&mut h, &creator, seat0, system_program::id())
        .expect_err("close_seat unexpectedly succeeded into builtin authority");
    assert!(err.contains("Error") || err.contains("Readonly") || err.contains("External"));

    let stuck_seat = get_seat(&h.svm, &seat0);
    assert_eq!(stuck_seat.status, SeatStatus::Open);
    assert_eq!(stuck_seat.authority, system_program::id());
    assert!(close_wager(&mut h).is_err());
}

```

Expected result:

- Self-authority either fails close while stuck or leaves a zeroed/nonzero-lamport seat account that blocks final `close_wager` cleanup.
- Protected authority cleanup fails and leaves the seat state unchanged.

Recommendation

The simplest patch is to require the payer to be the seat authority when taking a seat:

```

if dto.authority != *payer.key {
    return Err(ProgramError::InvalidArgument);
}

```

This makes the stored authority match the signer who paid to create or take the seat, so a caller cannot assign the seat to the seat PDA, the system program, or another unsafe recipient.

If delegated authorities are required later, the program should add an explicit delegated-authority flow. At minimum, the delegated authority account should be passed, should sign, and should not be the seat PDA or a protected recipient:

```
let authority_account = next_account_info(accounts_iter)?;

if *authority_account.key != dto.authority {
    return Err(ProgramError::InvalidArgument);
}

if !authority_account.is_signer {
    return Err(ProgramError::MissingRequiredSignature);
}

if dto.authority == *seat_account.key {
    return Err(ProgramError::InvalidArgument);
}

if *authority_account.key == system_program::ID {
    return Err(ProgramError::InvalidArgument);
}
```

As defense in depth, `close_seat` can also reject self-recipient closes before crediting lamports:

```
if *seat_authority.key == *seat_account.key {
    return Err(ProgramError::InvalidArgument);
}
```

Add regression tests that confirm `take_seat` rejects `dto.authority` values equal to the seat PDA and the system program.

Team Response

Fixed and Validated. Caveat: The latest code rejects the submitted harmful self-PDA and `system_program` authority cases, sends close rent to the stored `seat_authority`, and treats system-owned, data-empty seat PDAs as closed during `close_wager`. Non-creator callers must use themselves as seat authority, while creator-only reservations for another wallet remain intentionally allowed and should be documented as an accepted product caveat rather than as an unfixed arbitrary-authority path.

[I-05] Creator-only `append_terms` overwrite does not clear stale raw terms bytes

`append_terms` lets the contract creator write terms in chunks before the contract is finalized. The instruction also supports `overwrite=true`, which clears the in-memory `terms.content` string before appending the replacement chunk.

This path is creator-only and pre-finalization only:

```
let mut contract = Contract::from_account(contract_account)?;

if contract.creator != *payer.key {
    return Err(ProgramError::IllegalOwner);
}

if contract.finalized {
    return Err(ProgramError::InvalidAccountData);
}
```

The issue is that overwrite clears only the decoded Rust string:

```
if dto.overwrite {
    match &mut terms.content {
        Content::Plaintext(s) => s.clear(),
    }
}
```

After the replacement chunk is appended, the program saves the new `Terms` value into the existing account:

```
terms.save(terms_account)?;
```

The shared `save` helper serializes into the existing account buffer but does not clear the buffer first:

```
fn save(&self, account: &AccountInfo) -> Result<(), ProgramError> {
    let mut data = account.try_borrow_mut_data()?;
    self.serialize(&mut &mut data[..])
        .map_err(|_| ProgramError::InvalidAccountData)
}
```

If the creator overwrites longer draft terms with shorter draft terms, normal Borsh deserialization returns the new shorter terms, but stale bytes from the previous longer draft can remain in the raw account data after the new serialized object.

Relevant code paths:

- `contract_program/src/processor.rs:205` loads the contract
- `contract_program/src/processor.rs:207` restricts the call to the contract creator
- `contract_program/src/processor.rs:227` rejects finalized contracts
- `contract_program/src/processor.rs:231` handles overwrite mode
- `contract_program/src/processor.rs:270` saves the updated terms
- `protocol/src/traits.rs:25` serializes into the account buffer without zeroing it first

Impact

This is a stale raw-data cleanup issue, not a direct protocol security issue.

The public cannot overwrite another user's terms. The contract creator must trigger this path before finalization, and normal Borsh decoding should show only the new overwritten terms because the serialized string length is updated.

However, Solana account data is public raw bytes. If a creator overwrites a longer draft with a shorter draft, raw account inspection may still recover trailing bytes from the previous draft.

This can make overwrite semantics misleading for creators who expect the previous draft terms to be erased from account storage.

There is no direct fund loss, authorization bypass, payout impact, or settlement impact. This should be treated as Informational hardening.

Proof of Concept

The vulnerable behavior is visible from the write path:

```
if dto.overwrite {
  match &mut terms.content {
    Content::Plaintext(s) => s.clear(),
  }
}

match &mut terms.content {
  Content::Plaintext(s) => {
    let chunk_str = String::from_utf8(dto.chunk)
      .map_err(|_| ProgramError::InvalidInstructionData)?;
    s.push_str(&chunk_str);
  }
}

terms.save(terms_account)?;
```

Because save does not zero the account buffer, this sequence can leave stale suffix bytes:

```
old decoded terms: "old-sensitive-draft-terms"
new decoded terms: "new"

normal decoded value after overwrite:
"new"

possible raw account bytes after overwrite:
"new-sensitive-draft-terms"
```

A regression test can demonstrate the issue by overwriting longer terms with shorter terms and then scanning raw account bytes:

```
#[test]
fn append_terms_overwrite_shorter_content_leaves_old_raw_bytes() {
  let mut h = setup_terms_backed_contract(128);

  append_terms(
    &mut h,
    b"old-sensitive-draft-terms".to_vec(),
    false,
    false,
  )
  .unwrap();

  append_terms(
    &mut h,
    b"new".to_vec(),
    true,
    false,
  )
  .unwrap();

  let terms_account = h.svm.get_account(&h.terms).unwrap();
  let decoded = Terms::deserialize(&mut &terms_account.data[..]).unwrap();
```

```

assert_eq!(decoded.content.as_bytes(), b"new");
assert!(
    terms_account
        .data
        .windows(b"sensitive-draft".len())
        .any(|window| window == b"sensitive-draft"),
    "old overwritten terms remain recoverable from raw account data"
);
}

```

Expected result:

- normal `Terms` deserialization returns only the shorter replacement terms;
- raw account bytes can still contain a suffix from the previous longer draft.

Recommendation

Clear the terms account buffer before saving overwritten terms.

Minimal patch inside `append_terms`:

```

if dto.overwrite {
    terms_account.try_borrow_mut_data()?.fill(0);
}

terms.save(terms_account)?;

```

A cleaner shared fix is to add a zeroing save helper for variable-length account types:

```

pub fn save_zeroed(&self, account: &AccountInfo) -> Result<(), ProgramError> {
    let mut data = account.try_borrow_mut_data()?;
    data.fill(0);
    self.serialize(&mut &mut data[..])
        .map_err(|_| ProgramError::InvalidAccountData)
}

```

Then use it for overwrite paths where the new serialized value can be shorter than the old value:

```

if dto.overwrite {
    terms.save_zeroed(terms_account)?;
} else {
    terms.save(terms_account)?;
}

```

Add a regression test that overwrites long draft terms with shorter draft terms and asserts that the old raw substring is no longer present in the account data after the fix.

Team Response

Fixed and Validated. The Riverboat team added a `save_zeroed` serialization helper and now uses it when `append_terms` overwrites existing `Terms` content. The account data buffer is zeroed before the shorter replacement content is serialized, preventing stale trailing plaintext bytes from remaining in storage.

[I-06] set_contract_result can mark a contract resolved with a Pending result

set_contract_result lets the configured contract resolution authority submit a result for the parent contract. The instruction verifies that the submitted Judgment is valid for the contract's outcome type, but it does not verify that the submitted judgment is terminal.

For binary contracts, Judgment::is_valid accepts any Judgment::SingleOutcome(_) except out-of-range resolved values:

```
pub fn is_valid(&self, outcome_type: &OutcomeType) -> bool {
    match (outcome_type, self) {
        (OutcomeType::Binary, Judgment::SingleOutcome(EventResult::Resolved(k))) => *k <= 1,
        (OutcomeType::Binary, Judgment::SingleOutcome(_)) => true,
    }
}
```

Because Pending is also a SingleOutcome, the resolution authority can submit:

```
Judgment::SingleOutcome(EventResult::Pending)
```

set_contract_result then treats that value as a submitted result and updates the contract as resolved:

```
if !judgment.is_valid(&contract.outcomes) {
    return Err(ProgramError::InvalidArgument);
}

contract.resolved = true;
contract.result = Some(judgment);
contract.result_set_at = Some(Clock::get()?.unix_timestamp);
contract.save(contract_account)?;
```

This leaves the contract in a contradictory state:

```
resolved = true
result = Pending
result_set_at = Some(timestamp)
```

In other words, the account says the contract is resolved, but the stored result still says no outcome has been provided.

This state is not irreversible. The same resolution authority can later overwrite the stored Pending value because the overwrite guard only blocks already-settled results. The issue is that the program temporarily records an internally inconsistent parent state and starts result-dependent lifecycle timing before any terminal result has been submitted.

This is distinct from the unfinalized-contract lifecycle issue described in Finding L-01. That finding covers calling set_contract_result before contract.finalized == true. This issue applies even to finalized contracts: the submitted result itself can be non-terminal.

Relevant code paths:

- protocol/src/state.rs:307 defines Judgment::is_valid
- protocol/src/state.rs:310 accepts Judgment::SingleOutcome(_), including Pending
- protocol/src/state.rs:326 defines is_settled as resolved or push

- `contract_program/src/processor.rs:311` restricts the call to the resolution authority
- `contract_program/src/processor.rs:326` checks only `judgment.is_valid`
- `contract_program/src/processor.rs:330` sets `contract.resolved = true`
- `contract_program/src/processor.rs:331` stores the submitted judgment
- `contract_program/src/processor.rs:332` sets `result_set_at`
- `contract_program/src/processor.rs:371` starts the close window from `result_set_at`

Impact

The configured resolution authority can create an internally inconsistent parent state without submitting a terminal result such as `Resolved(0)`, `Resolved(1)`, or `Push`.

This does not create direct fund theft, and it does not give a public attacker new authority. The actor must already be the contract's resolution authority. The issue is lifecycle correctness: frontends, indexers, and users can observe `resolved = true` while the stored result is still `Pending`.

The inconsistent state also affects availability and cleanup semantics. New wager creation rejects resolved contracts:

```
if !contract.finalized || contract.resolved {  
    return Err(ProgramError::InvalidAccountData);  
}
```

The contract close window also starts from `result_set_at`, even though no terminal result was posted. If the `Pending` value is not corrected before the window elapses, the creator can close the contract while its stored result remains `Pending`.

This is Informational because the impact is bounded to lifecycle correctness and availability for the affected contract, the caller must already be the configured resolution authority, and the same authority can later correct the `Pending` value by submitting a terminal result.

Proof of Concept

PoC file:

```
contract_program/tests/test_set_contract_result_pending.rs
```

Run command:

```
cargo test -p contract_program --test test_set_contract_result_pending  
set_contract_result_accepts_pending_and_marks_contract_resolved -- --nocapture
```

Complete PoC code:


```

use borsh::{to_vec, BorshDeserialize};

use contract_program::dto::WriteContractData;
use contract_program::instruction::ContractInstruction;
use litesvm::LiteSVM;
use protocol::ids::CONTRACT_PROGRAM_ID;
use protocol::state::{Content, Contract, EventResult, Judgment, OutcomeType};
use solana_address::Address;
use solana_program::hash::hash;
use solana_sdk::{
    instruction::{AccountMeta, Instruction},
    message::Message,
    signature::{Keypair, Signer},
    transaction::Transaction,
};
use solana_sdk_ids::system_program;
use solana_sysvar::clock::Clock;
use std::time::{SystemTime, UNIX_EPOCH};

fn send_ok(svm: &mut LiteSVM, step: &str, tx: Transaction) {
    let meta = svm
        .send_transaction(tx)
        .unwrap_or_else(|err| panic!("{step} failed: {err:?}"));
    println!("{step} tx logs: {:#?}", meta.logs);
}

fn get_contract(svm: &LiteSVM, contract: &Address) -> Contract {
    let account = svm
        .get_account(contract)
        .expect("contract account should exist");

    Contract::deserialize(&mut &account.data[..])
        .expect("contract account should deserialize")
}

#[test]
fn set_contract_result_accepts_pending_and_marks_contract_resolved() {
    println!("=== PoC: set_contract_result accepts Pending as a contract result ===");

    let mut svm = LiteSVM::new();
    let program_id = CONTRACT_PROGRAM_ID;
    svm.add_program_from_file(program_id, "../target/deploy/contract_program.so")
        .unwrap();

    let mut clock = Clock::default();
    clock.unix_timestamp = SystemTime::now()
        .duration_since(UNIX_EPOCH)
        .unwrap()
        .as_secs() as i64;
    svm.set_sysvar(&clock);

```

```

let creator = Keypair::new();
let resolution_authority = Keypair::new();
svm.airdrop(&creator.pubkey(), 1_000_000_000).unwrap();
svm.airdrop(&resolution_authority.pubkey(), 1_000_000_000)
    .unwrap();

let headline = "Issue 11 Pending result PoC".to_string();
let headline_hash = hash(headline.as_bytes()).to_bytes();
let resolves_by = clock.unix_timestamp + (7 * 24 * 60 * 60);

let (contract_pda, _) = Address::find_program_address(
    &[
        b"contract",
        creator.pubkey().as_ref(),
        &headline_hash[..],
    ],
    &program_id,
);

let write_dto = WriteContractData {
    resolution_authority: Some(resolution_authority.pubkey()),
    resolves_by,
    private: false,
    category: [0u8; 8],
    hashtag: [0u8; 16],
    outcomes: OutcomeType::Binary,
    headline: Content::Plaintext(headline),
    terms_size: 0,
};

let write_ix = Instruction::new_with_bytes(
    program_id,
    &to_vec(&ContractInstruction::WriteContract { dto: write_dto }).unwrap(),
    vec![
        AccountMeta::new(creator.pubkey(), true),
        AccountMeta::new_readonly(system_program::id(), false),
        AccountMeta::new(contract_pda, false),
    ],
);

let write_tx = Transaction::new(
    [&creator],
    Message::new(&[write_ix], Some(&creator.pubkey())),
    svm.latest_blockhash(),
);

send_ok(&mut svm, "write finalized contract", write_tx);

let before = get_contract(&svm, &contract_pda);
println!(
    "before: finalized={}, resolved={}, result={:?}, result_set_at={:?}",
    before.finalized, before.resolved, before.result, before.result_set_at
);
assert!(before.finalized, "terms_size=0 should create a finalized contract");
assert!(!before.resolved, "contract should start unresolved");
assert_eq!(
    before.result,
    Some(Judgment::SingleOutcome(EventResult::Pending))
);
assert!(before.result_set_at.is_none());

```

```

let pending_result = Judgment::SingleOutcome(EventResult::Pending);
let set_result_ix = Instruction::new_with_bytes(
    program_id,
    &to_vec(&ContractInstruction::SetContractResult {
        judgment: pending_result,
    })
    .unwrap(),
    vec![
        AccountMeta::new_readonly(resolution_authority.pubkey(), true),
        AccountMeta::new(contract_pda, false),
    ],
);
let set_result_tx = Transaction::new(
    &[&resolution_authority],
    Message::new(&[set_result_ix], Some(&resolution_authority.pubkey())),
    svm.latest_blockhash(),
);
send_ok(&mut svm, "set Pending result", set_result_tx);

let after = get_contract(&svm, &contract_pda);
let stored_result = after
    .result
    .as_ref()
    .expect("result should still be present");

println!(
    "after: finalized={}, resolved={}, result={:?}, is_pending={}, is_settled={}, result_set_at={:?}"
    after.finalized,
    after.resolved,
    after.result,
    stored_result.is_pending(),
    stored_result.is_settled(),
    after.result_set_at
);

assert!(after.finalized, "contract remains finalized");
assert!(
    after.resolved,
    "BUG: set_contract_result marked the contract resolved"
);
assert_eq!(
    after.result,
    Some(Judgment::SingleOutcome(EventResult::Pending)),
    "BUG: stored result is still Pending"
);
assert!(
    !stored_result.is_settled(),
    "BUG: resolved contract has a non-terminal result"
);
assert!(
    after.result_set_at.is_some(),
    "BUG: result_set_at was stamped for a Pending result"
);

println!("=== PoC completed: Pending was accepted and recorded as resolved ===");
}

```

Observed result:

```

running 1 test
=== PoC: set_contract_result accepts Pending as a contract result ===
before: finalized=true, resolved=false, result=Some(SingleOutcome(Pending)), result_set_at=None
after: finalized=true, resolved=true, result=Some(SingleOutcome(Pending)), is_pending=true, is_settled=
      false, result_set_at=Some(1781538067)
=== PoC completed: Pending was accepted and recorded as resolved ===
test set_contract_result_accepts_pending_and_marks_contract_resolved ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out

```

The expected fixed behavior is that `Pending` is rejected and only terminal contract results are accepted.

Recommendation

Require submitted contract results to be terminal before setting `contract.resolved` OR `contract.result_set_at`.

Keep the existing shape check:

```

if !judgment.is_valid(&contract.outcomes) {
    return Err(ProgramError::InvalidArgument);
}

```

Then reject non-terminal results:

```

if !judgment.is_settled() {
    return Err(ProgramError::InvalidArgument);
}

```

The resulting check should be:

```

if !judgment.is_valid(&contract.outcomes) {
    return Err(ProgramError::InvalidArgument);
}

if !judgment.is_settled() {
    return Err(ProgramError::InvalidArgument);
}

```

If Riverboat intends parent contract results to reject `Push`, use `judgment.is_resolved()` instead:

```

if !judgment.is_resolved() {
    return Err(ProgramError::InvalidArgument);
}

```

Add regression tests for:

- `Pending` is rejected by `set_contract_result`;
- `Resolved(0)` is accepted;
- `Resolved(1)` is accepted;
- `Push` is either accepted or rejected according to the intended contract-level resolution policy.

Team Response

Fixed and Validated. The Riverboat team removed contract-level `result` and `result_set_at` fields and the contract result-setting path from the latest MVP design. A contract can no longer be marked resolved with a pending judgment through `set_contract_result`.